
SECTION NO 1: REGRESSION

TABLE OF CONTENTS

STEP 1	6
---------------	----------

INTRODUCTION TO PYTHON AND LIBRARIES FOR MACHINE LEARNING, ENVIRONMENTAL SETUP

STEP 2	7
---------------	----------

HANDLING MISSING VALUES, DATA NORMALIZATION, STANDARDIZATION, AND VISUALIZATION WITH MATPLOTLIB AND SEABORN

STEP 3	12
---------------	-----------

DATA PREPROCESSING – (CORRELATION, OUTLIER DETECTION, FEATURE ENGINEERING, DATA SPLITTING)

STEP 4	33
---------------	-----------

IMPLEMENTING LINEAR REGRESSION, MULTIPLE LINEAR REGRESSION

STEP 5	41
---------------	-----------

LOGISTIC REGRESSION FOR BINARY CLASSIFICATION & MODEL EVALUATION USING METRICS (R^2 , MAE, MSE)

SECTION NO 2: CLASSIFICATION

TABLE OF CONTENTS

STEP 6	65
---------------	-----------

IMPLEMENTING DECISION TREE

STEP 7 **66****IMPLEMENTING SUPPORT VECTOR MACHINE**

STEP 8 **68****EVALUATION METRICS FOR CLASSIFICATION (ACCURACY, CONFUSION MATRIX, AUC/ROC CURVE, PRECISION, RECALL, AND F1-SCORE)**

STEP 9 **72****TECHNIQUES FOR HANDLING IMBALANCED DATASETS (SMOTE, UNDER-SAMPLING, OVERSAMPLING)**

STEP 10 **72****IMPLEMENTING ARTIFICIAL NEURAL NETWORKS FOR CLASSIFICATION**

STEP 11 **76****IMPLEMENTING RECOMMENDER SYSTEM FROM SCRATCH**

STEP 12 **78****IMPLEMENTING ENSEMBLE LEARNING AND OPTIMIZATION**

STEP 13 **80****IMPLEMENTING Q-LEARNING FOR SIMPLE TASKS (REINFORCEMENT LEARNING)**

STEP 14 **82****GIT AND GITHUB FROM SCRATCH**

PROJECT NO 1: REGRESSION

PROJECT NO 1: AMES HOUSE PRICE PREDICTION

This Jupyter notebook presents a comprehensive machine learning workflow for predicting house prices in Ames, Iowa, using advanced regression techniques. The project demonstrates the end-to-end pipeline commonly used in tabular data competitions, such as those on Kaggle.

KEY STEPS AND WORKFLOW

1. INTRODUCTION TO DATASET

- The dataset contains detailed features about residential homes in Ames, Iowa.
- The target variable is SalePrice.
- Both train and test datasets are loaded and concatenated for consistent preprocessing.

2. EXPLORATORY DATA ANALYSIS (EDA)

- The notebook explores both numerical and categorical features.
- Statistical summaries and sample data (via `.head()`, `.tail()`, `.sample()`, `.info()`) provide insight into the data structure and missing values.
- The EDA phase includes:
 - Distribution of sale prices
 - Analysis of numerical and categorical variables
 - Investigation of feature correlations
 - Identification of missing values

3. DATA PREPROCESSING AND FEATURE ENGINEERING

- Handling missing values in both train and test sets.
- Outlier detection and treatment.
- Feature engineering:
 - Creating new variables
 - Label encoding of categorical features
 - Transforming numerical features to categorical (if appropriate)
- Addressing skewness and normalization of variables.
- Feature scaling (StandardScaler).
- One-hot encoding for categorical features.

4. MODELING

- Regularized regression models:
 - Ridge Regression
 - Lasso Regression
- Tree-based models:
 - XGBoost
 - LightGBM
- An ensemble (averaging) model is also constructed to leverage the strengths of multiple regressors.

5. CONCLUSION

- The project concludes with a summary of findings and modeling results.

LIBRARIES AND TOOLS USED

- pandas, numpy: Data manipulation and analysis
- matplotlib, seaborn: Data visualization
- scikit-learn: Preprocessing, regression models, and PCA
- imbalanced-learn (SMOTE): Handling class imbalance (though more common for classification tasks)
- xgboost, lightgbm: Gradient boosting machine learning libraries

DATA DETAILS

- 1460 rows $\times$ 80 columns in the training set
- 1459 rows $\times$ 79 columns in the test set
- 36 numerical and 43 categorical features are identified

NOTABLE POINTS

- The project is well-organized with a clear table of contents and markdown explanations.
- Emphasis is placed on preprocessing and engineering features to improve model performance.
- Both regularized linear models and ensemble tree-based methods are implemented.

SUMMARY:

This notebook thoroughly covers the process of predicting Ames house prices, from EDA through advanced modeling, making it a valuable reference for anyone interested in regression problems and machine learning best practices on tabular data.

STEP 1

INTRODUCTION TO PYTHON AND LIBRARIES FOR MACHINE LEARNING, ENVIRONMENTAL SETUP

To introduce Python basics and essential libraries used in Machine Learning, and to set up the working environment.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline
```

CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
import numpy as np	Imports NumPy for numerical operations.
import pandas as pd	Imports pandas for data manipulation.
import matplotlib.pyplot as plt	Imports Matplotlib for plotting graphs.
import seaborn as sns	Imports Seaborn for advanced data visualization.
import warnings	Imports the warnings module to manage warnings.
warnings.filterwarnings('ignore')	Suppresses warning messages for cleaner output.
%matplotlib inline	Ensures plots display within the Jupyter notebook.

SUMMARY OF OUTPUT:

This step sets up the Python environment by importing essential libraries for data analysis and visualization. It also configures the notebook to display plots inline and suppresses unnecessary warnings for a cleaner output.

STEP 2

HANDLING MISSING VALUES, DATA NORMALIZATION, STANDARDIZATION, AND VISUALIZATION WITH MATPLOTLIB AND SEABORN

Handling Missing Values involves filling (e.g., with mean/median) or dropping incomplete data to ensure model accuracy. **Data Normalization** scales values to a [0, 1] range; **Standardization** transforms data to have a mean of 0 and standard deviation of 1. **Visualization** with **Matplotlib** and **Seaborn** helps understand patterns, distributions, and relationships using plots like histograms, boxplots, and heatmaps.

CODE:

```
train_data = pd.read_csv('Data/train.csv', index_col='Id')
test_data = pd.read_csv('Data/test.csv', index_col='Id')

X_train = train_data.drop(['SalePrice'], axis=1)
y = train_data.SalePrice

X = pd.concat([X_train, test_data], axis=0)
```

CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
<code>pd.read_csv('Data/train.csv', index_col='Id')</code>	Loads training data and sets the 'Id' column as index.
<code>pd.read_csv('Data/test.csv', index_col='Id')</code>	Loads testing data similarly.
<code>train_data.drop(['SalePrice'], axis=1)</code>	Removes target column to get training features.
<code>train_data.SalePrice</code>	Selects the target variable (house prices).
<code>pd.concat([X_train, test_data], axis=0)</code>	Combines train and test feature data for unified processing.

CODE:

```
print("Train data's size: ", X_train.shape)
print("Test data's size: ", test_data.shape)
```

OUTPUT:

Train data's size: (1460, 79)
Test data's size: (1459, 79)

CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
<code>X_train.shape</code>	Returns the shape (rows, columns) of training set.
<code>test_data.shape</code>	Returns the shape of test dataset.

CODE:

```
numCols = list(X_train.select_dtypes(exclude='object').columns)
print(f"There are {len(numCols)} numerical features:\n", numCols)
```

OUTPUT:

There are 36 numerical features:

```
['MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual', 'OverallCond',
'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtFinSF1', 'BsmtFinSF2',
'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF',
'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath',
'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageYrBlt',
'GarageCars', 'GarageArea', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch',
'3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal', 'MoSold', 'YrSold']
```

CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
<code>select_dtypes(exclude='object')</code>	Selects only numerical columns.
<code>list(... .columns)</code>	Converts column names to a list.
<code>len(numCols)</code>	Returns the number of numerical features.

CODE:

```
catCols = list(X_train.select_dtypes(include='object').columns)
print(f"There are {len(catCols)} numerical features:\n", catCols)
```

OUTPUT:

There are 43 numerical features:

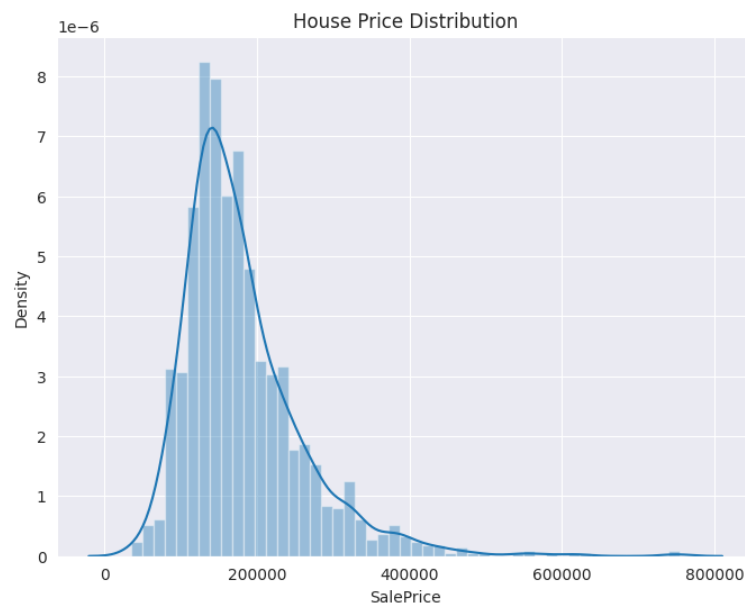
['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2', 'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenQual', 'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual', 'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature', 'SaleType', 'SaleCondition']

CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
<code>select_dtypes(include='object')</code>	Selects only categorical columns.
<code>list(... .columns)</code>	Converts column names to a list.
<code>len(catCols)</code>	Returns the number of categorical features.

CODE:

```
plt.figure(figsize=(8,6))
sns.distplot(y)
title = plt.title("House Price Distribution")
```

OUTPUT:**CODE EXPLANATION:**

CODE LINE	SIMPLE EXPLANATION
<code>plt.figure(figsize=(8,6))</code>	Sets figure size for the plot.
<code>sns.distplot(y)</code>	Plots distribution of the target variable (house price).
<code>plt.title(...)</code>	Adds a title to the graph.

CODE:

```
print(f"""Skewness: {y.skew()}
Kurtosis: {y.kurt()}""")
```

OUTPUT:

Skewness: 1.8828757597682129
Kurtosis: 6.536281860064529

CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
<code>y.skew()</code>	Calculates how skewed the price distribution is.
<code>y.kurt()</code>	Calculations peakedness of the price distribution.

SUMMARY OF OUTPUT:

In this step, the training and testing datasets are loaded, and basic structure checks are performed. The code identifies numerical and categorical features and visualizes the target variable's distribution. It also gives skewness and kurtosis to understand the data spread and shape.

STEP 3

DATA PREPROCESSING – (CORRELATION, OUTLIER DETECTION, FEATURE ENGINEERING, DATA SPLITTING)

Data preprocessing is a crucial step in any machine learning project. It involves preparing and cleaning the raw data before feeding it into a machine learning model. This step typically includes handling missing values, detecting and removing outliers, encoding categorical variables, normalizing or standardizing numerical features, and engineering new features. Proper preprocessing improves the quality of the data and helps machine learning models achieve better performance.

LOADING AND INSPECTING THE DATA

CODE:

```
train_data = pd.read_csv('Data/train.csv', index_col='Id')
test_data = pd.read_csv('Data/test.csv', index_col='Id')
```

CODE LINE		SIMPLE EXPLANATION
train_data pd.read_csv('Data/train.csv', index_col='Id')	=	Loads the training data from a CSV file using the 'Id' column as the index.
test_data pd.read_csv('Data/test.csv', index_col='Id')	=	Loads the test data using the 'Id' column as the index.

CODE:

```
X_train = train_data.drop(['SalePrice'], axis=1)
y = train_data.SalePrice
X = pd.concat([X_train, test_data], axis=0)
```


CODE LINE	SIMPLE EXPLANATION
<code>X_train = train_data.drop(['SalePrice'], axis=1)</code>	Removes the 'SalePrice' column from training data to get input features.
<code>y = train_data.SalePrice</code>	Sets the target variable as the 'SalePrice' column.
<code>X = pd.concat([X_train, test_data], axis=0)</code>	Combines training and test features for preprocessing.

CODE:

```
print("Train data's size: ", X_train.shape)
print("Test data's size: ", test_data.shape)
```

CODE LINE	SIMPLE EXPLANATION
<code>print("Train data's size: ", X_train.shape)</code>	Prints the shape (rows, columns) of the training data features.
<code>print("Test data's size: ", test_data.shape)</code>	Prints the shape of the test data.

OUTPUT:

```
Train data's size: (1460, 79)
Test data's size: (1459, 79)
```

IDENTIFYING NUMERICAL AND CATEGORICAL FEATURES**CODE:**

```
numCols = list(X_train.select_dtypes(exclude='object').columns)
print(f"There are {len(numCols)} numerical features:\n", numCols)
```

CODE LINE	SIMPLE EXPLANATION
<code>numCols = list(X_train.select_dtypes(exclude='object').columns)</code>	Finds all columns with numeric data types.

<code>print(f"There are {len(numCols)} numerical features:\n", numCols)</code>	Prints the count and names of numeric features.
--	---

OUTPUT:

There are 36 numerical features:

```
['MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual', 'OverallCond',
'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtFinSF1', 'BsmtFinSF2',
'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF',
'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath',
'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageYrBlt',
'GarageCars', 'GarageArea', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch',
'3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal', 'MoSold', 'YrSold']
```

CODE:

```
catCols = list(X_train.select_dtypes(include='object').columns)
print(f"There are {len(catCols)} numerical features:\n", catCols)
```

CODE LINE	SIMPLE EXPLANATION
<code>catCols = list(X_train.select_dtypes(include='object').columns)</code>	Finds all columns with categorical (object) data types.
<code>print(f"There are {len(catCols)} numerical features:\n", catCols)</code>	Prints the count and names of categorical features.

OUTPUT:

There are 43 numerical features:

```
['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities',
'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2',
'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st',
'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundation',
'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2',
'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenQual',
'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual',
'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature', 'SaleType',
'SaleCondition']
```

CODE:

```

train_data.head()
train_data.tail()
train_data.shape
train_data.sample()

```

CODE LINE	SIMPLE EXPLANATION
<code>train_data.head()</code>	Displays the first five rows of the training data.
<code>train_data.tail()</code>	Displays the last five rows of the training data.
<code>train_data.shape</code>	Shows the shape (rows, columns) of the training data.
<code>train_data.sample()</code>	Shows a random row from the training data.

OUTPUT:

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	LotConfig	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType
Id																			
1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	Inside	...	0	NaN	NaN	NaN	0	2	2008	WD
2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	FR2	...	0	NaN	NaN	NaN	0	5	2007	WD
3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	Inside	...	0	NaN	NaN	NaN	0	9	2008	WD
4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	Corner	...	0	NaN	NaN	NaN	0	2	2006	WD
5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	FR2	...	0	NaN	NaN	NaN	0	12	2008	WD

5 rows × 80 columns

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	LotConfig	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType
Id																			
1456	60	RL	62.0	7917	Pave	NaN	Reg	Lvl	AllPub	Inside	...	0	NaN	NaN	NaN	0	8	2007	WD
1457	20	RL	85.0	13175	Pave	NaN	Reg	Lvl	AllPub	Inside	...	0	NaN	MnPrv	NaN	0	2	2010	WD
1458	70	RL	66.0	9042	Pave	NaN	Reg	Lvl	AllPub	Inside	...	0	NaN	GdPrv	Shed	2500	5	2010	WD
1459	20	RL	68.0	9717	Pave	NaN	Reg	Lvl	AllPub	Inside	...	0	NaN	NaN	NaN	0	4	2010	WD
1460	20	RL	75.0	9937	Pave	NaN	Reg	Lvl	AllPub	Inside	...	0	NaN	NaN	NaN	0	6	2008	WD

5 rows × 80 columns

(1460, 80)

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	LotConfig	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType
Id																			
1388	50	RM	60.0	8520	Pave	Grlv	Reg	Lvl	AllPub	Inside	...	0	NaN	GdWo	NaN	0	8	2007	CWD

1 rows × 80 columns

CODE:

```
train_data.info()
train_data.describe()
train_data.isnull().sum()
```

CODE LINE	SIMPLE EXPLANATION
<code>train_data.info()</code>	Prints information about data types and non-null values in each column.
<code>train_data.describe()</code>	Provides statistics (mean, std, min, etc.) for numeric columns.
<code>train_data.isnull().sum()</code>	Counts missing values in each column.

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
Index: 1460 entries, 1 to 1460
Data columns (total 80 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   MSSubClass      1460 non-null   int64
1   MSZoning        1460 non-null   object
2   LotFrontage     1201 non-null   float64
3   LotArea         1460 non-null   int64
4   Street         1460 non-null   object
5   Alley          91 non-null     object
6   LotShape        1460 non-null   object
7   LandContour     1460 non-null   object
8   Utilities       1460 non-null   object
9   LotConfig       1460 non-null   object
10  Landslope       1460 non-null   object
11  Neighborhood    1460 non-null   object
12  Condition1      1460 non-null   object
13  Condition2      1460 non-null   object
14  BldgType        1460 non-null   object
15  HouseStyle      1460 non-null   object
16  OverallQual     1460 non-null   int64
17  OverallCond     1460 non-null   int64
18  YearBuilt       1460 non-null   int64
19  YearRemodAdd    1460 non-null   int64
...
78  SaleCondition   1460 non-null   object
79  SalePrice       1460 non-null   int64
dtypes: float64(3), int64(34), object(43)
memory usage: 923.9+ KB

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	BsmtFinSF2	...	WoodDeckSF	OpenPorchSF	EnclosedPorch
count	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1452.000000	1460.000000	1460.000000	...	1460.000000	1460.000000	1460.000000
mean	56.897260	70.049958	10516.828082	6.099315	5.575342	1971.267808	1984.865753	103.685262	443.639726	46.549315	...	94.244521	46.660274	21.954110
std	42.300571	24.284752	9981.264932	1.382997	1.112799	30.202904	20.645407	181.066207	456.098091	161.319273	...	125.338794	66.256028	61.119149
min	20.000000	21.000000	1300.000000	1.000000	1.000000	1872.000000	1950.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000
25%	20.000000	59.000000	7553.500000	5.000000	5.000000	1954.000000	1967.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000
50%	50.000000	69.000000	9478.500000	6.000000	5.000000	1973.000000	1994.000000	0.000000	383.500000	0.000000	...	0.000000	25.000000	0.000000
75%	70.000000	80.000000	11601.500000	7.000000	6.000000	2000.000000	2004.000000	166.000000	712.250000	0.000000	...	168.000000	68.000000	0.000000
max	190.000000	313.000000	215245.000000	10.000000	9.000000	2010.000000	2010.000000	1600.000000	5644.000000	1474.000000	...	857.000000	547.000000	552.000000

8 rows × 37 columns

```
MSSubClass      0
MSZoning        0
LotFrontage     259
LotArea         0
Street          0
...
MoSold          0
YrSold          0
SaleType        0
SaleCondition    0
SalePrice       0
Length: 80, dtype: int64
```

HANDLING MISSING VALUES

CODE:

```
numeric_cols = train_data.select_dtypes(include=[np.number])
non_numeric_cols = train_data.select_dtypes(exclude=[np.number])
```

CODE LINE		SIMPLE EXPLANATION
numeric_cols train_data.select_dtypes(include=[np.number])	=	Selects only the numeric columns from training data.
non_numeric_cols train_data.select_dtypes(exclude=[np.number])	=	Selects only the non-numeric (categorical) columns.

CODE:

```
numeric_cols.fillna(numeric_cols.mean(), inplace=True)
train_data = pd.concat([numeric_cols, non_numeric_cols], axis=1)
```

CODE LINE	SIMPLE EXPLANATION
numeric_cols.fillna(numeric_cols.mean(), inplace=True)	Fills missing numeric values with their mean.
train_data = pd.concat([numeric_cols, non_numeric_cols], axis=1)	Combines numeric and non-numeric columns back into one DataFrame.

CODE:

```

missing_values = train_data.isnull().sum()
print(missing_values)

```

CODE LINE	SIMPLE EXPLANATION
<code>missing_values = train_data.isnull().sum()</code>	= Counts missing values in each column after filling.
<code>print(missing_values)</code>	Prints the number of missing values per column.

OUTPUT:

```

MSSubClass      0
LotFrontage     0
LotArea         0
OverallQual     0
OverallCond     0
...
PoolQC         1453
Fence          1179
MiscFeature    1406
SaleType       0
SaleCondition  0
Length: 80, dtype: int64

```

CODE:

```

numeric_cols = train_data.select_dtypes(include=[np.number])
non_numeric_cols = train_data.select_dtypes(exclude=[np.number])
for col in non_numeric_cols.columns:
    non_numeric_cols[col].fillna(non_numeric_cols[col].mode()[0],
    inplace=True)

```

CODE LINE	SIMPLE EXPLANATION
<code>numeric_cols = train_data.select_dtypes(include=[np.number])</code>	= Re-selects numeric columns after previous operations.
<code>non_numeric_cols = train_data.select_dtypes(exclude=[np.number])</code>	= Re-selects non-numeric columns.
<code>for col in non_numeric_cols.columns:</code>	Iterates over each non-numeric column.

```
non_numeric_cols[col].fillna  
(non_numeric_cols[col].mode(  
)[0], inplace=True)
```

Fills missing categorical values with the most frequent value (mode) of each column.

OUTPUT:

```
MSSubClass      0  
LotFrontage     0  
LotArea         0  
OverallQual     0  
OverallCond     0  
..  
PoolQC          0  
Fence           0  
MiscFeature     0  
SaleType        0  
SaleCondition   0  
Length: 80, dtype: int64
```

CODE:

```
train_data = pd.concat([numeric_cols, non_numeric_cols], axis=1)  
missing_values = train_data.isnull().sum()  
print(missing_values)
```

CODE LINE		SIMPLE EXPLANATION
train_data pd.concat([numeric_cols, non_numeric_cols], axis=1)	=	Recombines the columns into a single dataset.
missing_values train_data.isnull().sum()	=	Counts remaining missing values after filling all.
print(missing_values)		Displays the final missing value count.

OUTPUT:

```
MSSubClass      0  
LotFrontage     0  
LotArea         0  
OverallQual     0  
OverallCond     0  
..  
PoolQC          0  
Fence           0  
MiscFeature     0  
SaleType        0  
SaleCondition   0  
Length: 80, dtype: int64
```

DROPPING REMAINING NULLS AND DUPLICATES

CODE:

```
train_data.isnull().sum()
train_data.dropna(inplace=True)
```

CODE LINE	SIMPLE EXPLANATION
train_data.isnull().sum()	Checks for any remaining missing values.
train_data.dropna(inplace=True)	Removes all rows that contain any missing values.

OUTPUT:

```
MSSubClass      0
LotFrontage     0
LotArea         0
OverallQual     0
OverallCond     0
..
PoolQC         0
Fence         0
MiscFeature    0
SaleType      0
SaleCondition  0
Length: 80, dtype: int64
```

CODE:

```
missing_values = train_data.isnull().sum()
print(missing_values)
```

CODE LINE	SIMPLE EXPLANATION
missing_values = train_data.isnull().sum()	Re-checks the dataset for missing values after dropping.
print(missing_values)	Prints the count of missing values per column. (Should be 0 now)

CODE:

```
train_data.drop_duplicates(inplace=True)
train_data.shape
```

CODE LINE	SIMPLE EXPLANATION
<code>train_data.drop_duplicates(inplace=True)</code>	Removes any duplicate rows from the dataset.
<code>train_data.shape</code>	Displays the shape of the dataset after dropping duplicates.

OUTPUT:

```
(1460, 80)
```

DETECTING AND REMOVING OUTLIERS USING IQR**CODE:**

```
numeric_cols = train_data.select_dtypes(include=[np.number])
Q1 = numeric_cols.quantile(0.25)
Q3 = numeric_cols.quantile(0.75)
IQR = Q3 - Q1
```

CODE LINE	SIMPLE EXPLANATION
<code>numeric_cols = train_data.select_dtypes(include=[np.number])</code>	Selects numeric columns for outlier detection.
<code>Q1 = numeric_cols.quantile(0.25)</code>	Calculates the first quartile (25th percentile) for numeric features.
<code>Q3 = numeric_cols.quantile(0.75)</code>	Calculates the third quartile (75th percentile).
<code>IQR = Q3 - Q1</code>	Computes the Interquartile Range (IQR) to help detect outliers.

CODE:

```

train_data_cleaned = train_data[~((numeric_cols < (Q1 - 1.5 * IQR)) |
                                   (numeric_cols > (Q3 + 1.5 *
IQR))).any(axis=1)]

```

CODE LINE	SIMPLE EXPLANATION
train_data_cleaned = ...	Removes rows with outlier values beyond $1.5 \times \text{IQR}$ from Q1 or Q3.

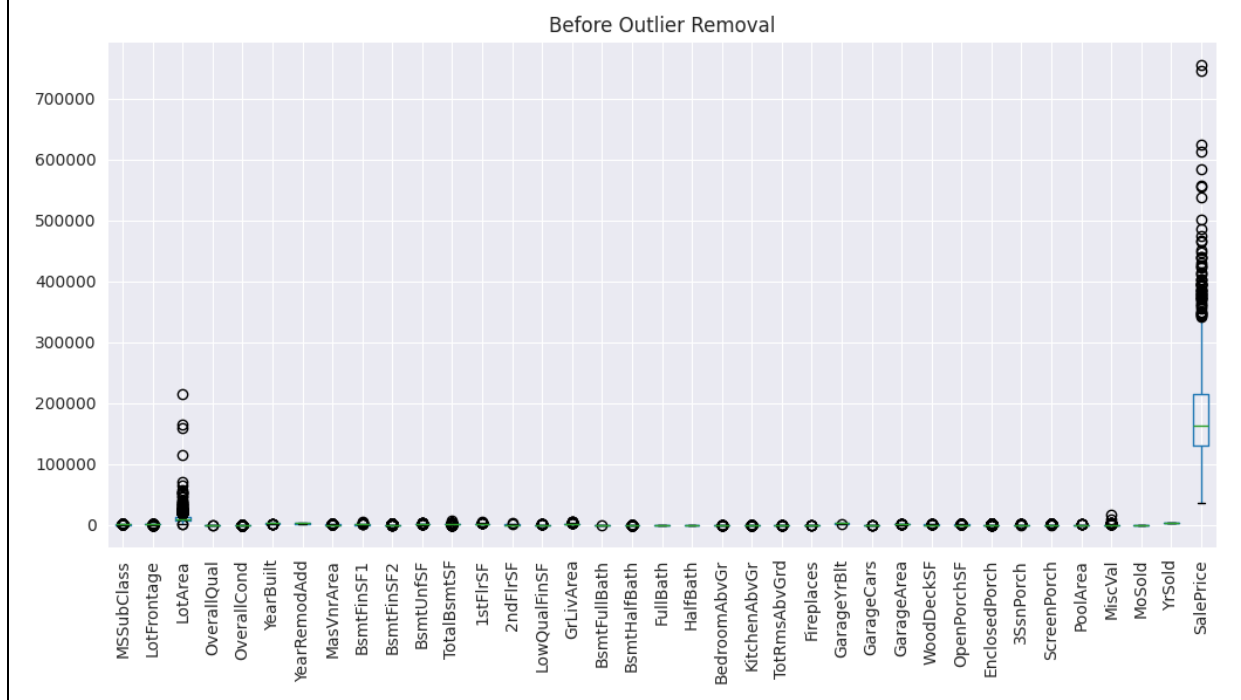
VISUALIZING OUTLIERS BEFORE AND AFTER REMOVAL**CODE:**

```

plt.figure(figsize=(20, 6))
plt.subplot(1, 2, 1)
numeric_cols.boxplot()
plt.title("Before Outlier Removal")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()

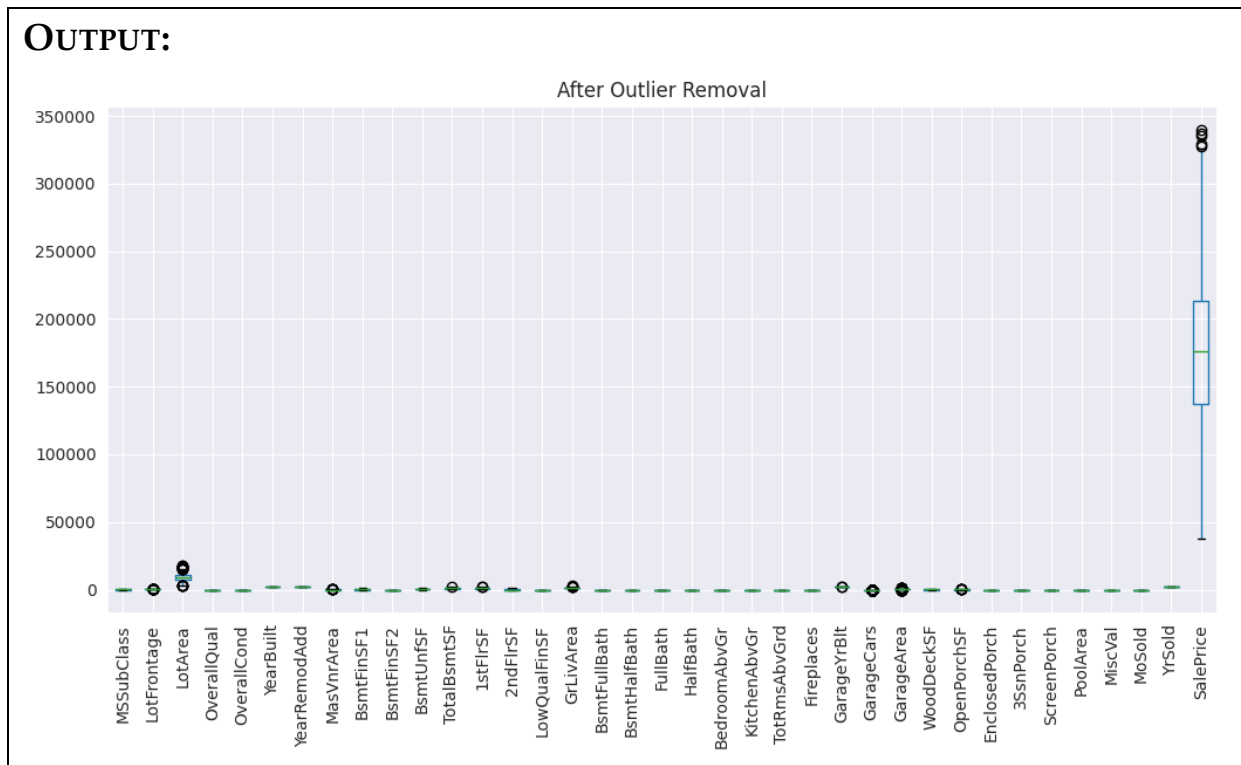
```

CODE LINE	SIMPLE EXPLANATION
plt.figure(figsize=(20, 6))	Creates a figure with specified size.
plt.subplot(1, 2, 1)	Prepares the first subplot in a 1-row, 2-column layout.
numeric_cols.boxplot()	Draws a boxplot of numeric features before outlier removal.
plt.title("Before Outlier Removal")	Adds a title to the plot.
plt.xticks(rotation=90)	Rotates x-axis labels for better readability.
plt.tight_layout()	Adjusts subplot layout for better spacing.
plt.show()	Displays the plot.

OUTPUT:**CODE:**

```
plt.figure(figsize=(20, 6))
plt.subplot(1, 2, 2)
train_data_cleaned.select_dtypes(include=[np.number]).boxplot()
plt.title("After Outlier Removal")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

CODE LINE	SIMPLE EXPLANATION
<code>plt.figure(figsize=(20, 6))</code>	Creates a new figure for the next boxplot.
<code>plt.subplot(1, 2, 2)</code>	Prepares the second subplot.
<code>train_data_cleaned.select_dtypes(include=[np.number]).boxplot()</code>	Boxplot of numeric data after outlier removal.
<code>plt.title("After Outlier Removal")</code>	Adds a title for the cleaned dataset.
<code>plt.xticks(rotation=90)</code>	Rotates x-axis labels.
<code>plt.tight_layout()</code>	Adjusts spacing.
<code>plt.show()</code>	Displays the plot.

OUTPUT:**NORMALIZING DATA WITH MINMAXSCALER****CODE:**

```
from sklearn.preprocessing import MinMaxScaler
numeric_cols = train_data.select_dtypes(include=[np.number])
non_numeric_cols = train_data.select_dtypes(exclude=[np.number])
scaler = MinMaxScaler()
scaled_numeric_data = scaler.fit_transform(numeric_cols)
```

CODE LINE	SIMPLE EXPLANATION
<code>from sklearn.preprocessing import MinMaxScaler</code>	Imports the MinMaxScaler class from scikit-learn.
<code>numeric_cols = train_data.select_dtypes(include=[np.number])</code>	Selects numeric columns for normalization.
<code>non_numeric_cols = train_data.select_dtypes(exclude=[np.number])</code>	Selects non-numeric columns to preserve separately.
<code>scaler = MinMaxScaler()</code>	Initializes the MinMaxScaler to scale features to a 0–1 range.

<code>scaled_numeric_data</code> <code>scaler.fit_transform(numeric_cols)</code>	=	Applies MinMaxScaler to numeric columns.
---	---	--

CODE:

```
scaled_numeric_df = pd.DataFrame(scaled_numeric_data,
                                columns=numeric_cols.columns)
scaled_data = pd.concat([scaled_numeric_df,
                        non_numeric_cols.reset_index(drop=True)], axis=1)
print(scaled_data.shape)
print()
print('*' * 60)
scaled_data.head()
```

CODE LINE	SIMPLE EXPLANATION
<code>scaled_numeric_df</code> <code>pd.DataFrame(scaled_numeric_data,</code> <code>columns=numeric_cols.columns)</code>	= Converts scaled numeric data back into a DataFrame.
<code>scaled_data</code> <code>pd.concat([scaled_numeric_df,</code> <code>non_numeric_cols.reset_index(drop=True)],</code> <code>axis=1)</code>	= Combines scaled numeric and original categorical data.
<code>print(scaled_data.shape)</code>	Displays the shape of the combined DataFrame.
<code>print()</code>	Prints a blank line for spacing.
<code>print('*' * 60)</code>	Prints a line of asterisks for visual separation.
<code>scaled_data.head()</code>	Displays the first five rows of the scaled dataset.

OUTPUT:

	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	BsmtFinSF2	...	GarageType	GarageFinish	GarageQual	GarageCond
0	0.235294	0.150685	0.033420	0.666667	0.500	0.949275	0.883333	0.12250	0.125089	0.0	...	Attchd	RFn	TA	TA
1	0.000000	0.202055	0.038795	0.555556	0.875	0.753623	0.433333	0.00000	0.173281	0.0	...	Attchd	RFn	TA	TA
2	0.235294	0.160959	0.046507	0.666667	0.500	0.934783	0.866667	0.10125	0.086109	0.0	...	Attchd	RFn	TA	TA
3	0.294118	0.133562	0.038561	0.666667	0.500	0.311594	0.333333	0.00000	0.038271	0.0	...	Detchd	Unf	TA	TA
4	0.235294	0.215753	0.060576	0.777778	0.500	0.927536	0.833333	0.21875	0.116052	0.0	...	Attchd	RFn	TA	TA

STANDARDIZING DATA WITH STANDARDSCALER**CODE:**

```

from sklearn.preprocessing import StandardScaler
numeric_cols = train_data.select_dtypes(include=[np.number])
non_numeric_cols = train_data.select_dtypes(exclude=[np.number])
scaler = StandardScaler()
scaled_numeric_data = scaler.fit_transform(numeric_cols)

```

CODE LINE	SIMPLE EXPLANATION
from sklearn.preprocessing import StandardScaler	Imports StandardScaler from scikit-learn.
numeric_cols = train_data.select_dtypes(include=[np.number])	Selects numeric columns to standardize.
non_numeric_cols = train_data.select_dtypes(exclude=[np.number])	Selects non-numeric columns to preserve.
scaler = StandardScaler()	Initializes the StandardScaler (mean = 0, std = 1).
scaled_numeric_data = scaler.fit_transform(numeric_cols)	Standardizes numeric columns.

CODE:

```

scaled_numeric_df = pd.DataFrame(scaled_numeric_data,
                                columns=numeric_cols.columns)
scaled_data = pd.concat([scaled_numeric_df,
                        non_numeric_cols.reset_index(drop=True)], axis=1)
print(scaled_data.shape)
print()
print('*' * 60)
scaled_data.head()

```

CODE LINE	SIMPLE EXPLANATION
scaled_numeric_df = pd.DataFrame(scaled_numeric_data, columns=numeric_cols.columns)	Converts standardized data back to DataFrame.
scaled_data = pd.concat([scaled_numeric_df, non_numeric_cols.reset_index(drop=True)], axis=1)	Combines standardized numeric and original categorical data.

<code>print(scaled_data.shape)</code>	Prints the shape of the standardized DataFrame.
<code>print()</code>	Adds a blank line.
<code>print('*' * 60)</code>	Prints separator for clarity.
<code>scaled_data.head()</code>	Displays the first five rows of standardized data.

OUTPUT:

	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	BsmtFinSF2	...	GarageType	GarageFinish	GarageQual	GarageCond
0	0.073375	-0.229372	-0.207142	0.651479	-0.517200	1.050994	0.878668	0.511418	0.575425	-0.288653	...	Attchd	Rfn	TA	TA
1	-0.872563	0.451936	-0.091886	-0.071836	2.179628	0.156734	-0.429577	-0.574410	1.171992	-0.288653	...	Attchd	Rfn	TA	TA
2	0.073375	-0.093110	0.073480	0.651479	-0.517200	0.984752	0.830215	0.323060	0.092907	-0.288653	...	Attchd	Rfn	TA	TA
3	0.309859	-0.456474	-0.096897	0.651479	-0.517200	-1.863632	-0.720298	-0.574410	-0.499274	-0.288653	...	Detchd	Unf	TA	TA
4	0.073375	0.633618	0.375148	1.374795	-0.517200	0.951632	0.733308	1.364570	0.463568	-0.288653	...	Attchd	Rfn	TA	TA

EXPLORING AND ENCODING CATEGORICAL VARIABLES**CODE:**

```
train_data["LandContour"].unique()
train_data.Neighborhood.unique()
```

CODE LINE	SIMPLE EXPLANATION
<code>train_data["LandContour"].unique()</code>	Displays all unique values in the "LandContour" categorical column.
<code>train_data.Neighborhood.unique()</code>	Displays all unique values in the "Neighborhood" categorical column.

OUTPUT:

```
array(['Lvl', 'Bnk', 'Low', 'HLS'], dtype=object)
array(['CollgCr', 'Veenker', 'Crawfor', 'NoRidge', 'Mitchel', 'Somerst',
       'NWAmes', 'OldTown', 'BrkSide', 'Sawyer', 'NridgHt', 'NAmes',
       'SawyerW', 'IDOTRR', 'MeadowV', 'Edwards', 'Timber', 'Gilbert',
       'StoneBr', 'ClearCr', 'NPKVill', 'Blmngtn', 'BrDale', 'SWISU',
       'Blueste'], dtype=object)
```

CODE:

```

from sklearn.preprocessing import StandardScaler
cat_features = [feature for feature in train_data.columns if
train_data[feature].dtype == 'O']
data1 = pd.get_dummies(cat_features)
data1

```

CODE LINE	SIMPLE EXPLANATION
<code>from sklearn.preprocessing import StandardScaler</code>	Re-imports StandardScaler (already imported earlier).
<code>cat_features = [feature for feature in train_data.columns if train_data[feature].dtype == 'O']</code>	Identifies all categorical features by checking for object data type.
<code>data1 = pd.get_dummies(cat_features)</code>	Incorrect: Attempts to one-hot encode a list instead of a DataFrame.
<code>data1</code>	Displays the resulting variable (will throw an error due to the previous line).

OUTPUT:

	Alley	BldgType	BsmtCond	BsmtExposure	BsmtFinType1	BsmtFinType2	BsmtQual	CentralAir	Condition1	Condition2	...	MiscFeature	Neighborhood	PavedDrive	PoolQC	RoofMatl	RoofStyle	SaleCondition	SaleType	Street	Utilities
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	True	False
2	True	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
5	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	True
6	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
7	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
8	False	False	False	False	False	False	False	False	False	False	...	False	True	False	False	False	False	False	False	False	False
9	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	False	False	False	False	False	False
10	False	False	False	False	False	False	False	False	False	True	...	False	False	False	False	False	False	False	False	False	False
11	False	True	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
12	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
13	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	True	False	False	False	False
14	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	True	False	False	False	False	False
15	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
16	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
17	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
18	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
19	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
20	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
21	False	False	False	False	False	False	True	False	False	False	...	False	False	False	False	False	False	False	False	False	False
22	False	False	True	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
23	False	False	False	True	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
24	False	False	False	False	True	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
25	False	False	False	False	False	True	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
26	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
27	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
28	False	False	False	False	False	False	False	True	False	False	...	False	False	False	False	False	False	False	False	False	False
29	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
30	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
31	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
32	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
33	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
34	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
35	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
36	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
37	False	False	False	False	False	False	False	False	False	False	...	False	True	False	False	False	False	False	False	False	False
38	False	False	False	False	False	False	False	False	False	False	...	False	False	True	False	False	False	False	False	False	False
39	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
40	False	False	False	False	False	False	False	False	False	False	...	True	False	False	False	False	False	False	False	False	False
41	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	True	False	False
42	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	True	False	False	False

43 rows x 43 columns

CODE:

```
cat_features = [feature for feature in train_data.columns if
train_data[feature].dtype == 'O']
data1 = pd.get_dummies(train_data, columns=cat_features)
scaled_data = pd.concat([train_data, data1], axis=1)
print(scaled_data.shape)
print()
print('*' * 70)
scaled_data.head()
```

CODE LINE	SIMPLE EXPLANATION
<code>cat_features = [feature for feature in train_data.columns if train_data[feature].dtype == 'O']</code>	Re-identifies categorical columns in the dataset.
<code>data1 = pd.get_dummies(train_data, columns=cat_features)</code>	Correctly one-hot encodes the categorical columns in the DataFrame.
<code>scaled_data = pd.concat([train_data, data1], axis=1)</code>	Combines the original and one-hot encoded data into a single DataFrame.
<code>print(scaled_data.shape)</code>	Prints the shape of the combined dataset.
<code>print()</code>	Adds spacing.
<code>print('*' * 70)</code>	Prints a visual separator.
<code>scaled_data.head()</code>	Shows the first few rows of the updated dataset.

OUTPUT:

(1460, 368)

	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	BsmtFinSF2	...	SaleType_ConLw	SaleType_New	SaleType_Oth
Id														
1	60	65.0	8450	7	5	2003	2003	196.0	706	0	...	False	False	False
2	20	80.0	9600	6	8	1976	1976	0.0	978	0	...	False	False	False
3	60	68.0	11250	7	5	2001	2002	162.0	486	0	...	False	False	False
4	70	60.0	9550	7	5	1915	1970	0.0	216	0	...	False	False	False
5	60	84.0	14260	8	5	2000	2000	350.0	655	0	...	False	False	False

5 rows x 368 columns

APPLYING PCA FOR DIMENSIONALITY REDUCTION

CODE:

```

from sklearn.decomposition import PCA
train_data.fillna(train_data.mean(numeric_only=True), inplace=True)
cat_features = [feature for feature in train_data.columns if
train_data[feature].dtype == 'O']
numeric_features = [feature for feature in train_data.columns if
train_data[feature].dtype != 'O']
train_data = pd.get_dummies(train_data, columns=cat_features)

```

CODE LINE	SIMPLE EXPLANATION
from sklearn.decomposition import PCA	Imports PCA from scikit-learn for dimensionality reduction.
train_data.fillna (train_data.mean(numeric_only=True), inplace=True)	Fills any remaining missing numeric values with their column means.
cat_features = [feature for feature in train_data.columns if train_data[feature].dtype == 'O']	Finds categorical columns to be encoded.
numeric_features = [feature for feature in train_data.columns if train_data[feature].dtype != 'O']	Lists all numeric columns (after encoding).
train_data = pd.get_dummies(train_data, columns=cat_features)	One-hot encodes the categorical features.

CODE:

```

scaler = StandardScaler()
train_data[numeric_features] =
scaler.fit_transform(train_data[numeric_features].values)
pca = PCA(n_components=15)
train_data_pca = pca.fit_transform(train_data)
print(train_data_pca.shape)
print(train_data_pca[:5])

```

CODE LINE	SIMPLE EXPLANATION
scaler = StandardScaler()	Initializes the StandardScaler for feature scaling.
train_data[numeric_features]	Standardizes numeric features to have zero mean and unit variance.

<code>= scaler.fit_transform(train_data[numeric_features].values)</code>	
<code>pca = PCA(n_components=15)</code>	Sets up PCA to reduce data to 15 principal components.
<code>train_data_pca = pca.fit_transform(train_data)</code>	Applies PCA to transform the original data.
<code>print(train_data_pca.shape)</code>	Prints the shape of the transformed dataset.
<code>print(train_data_pca[:5])</code>	Prints the first five PCA-transformed rows.

OUTPUT:

```
(1460, 15)
[[ 2.18069803  0.36951228 -2.15980227 -1.89760966  0.23839871 -0.31065059
 -1.49158063 -1.28499977 -0.19313676  0.01851965  0.60857621 -0.35480474
 -0.40396889  0.76324315 -0.62158832]
 [ -0.07589801 -1.24247227  1.51923852 -0.3402938  -1.60426727 -0.60899497
  0.33881534  2.40055008  1.94390033 -1.54102478 -0.98880322 -1.93608368
  0.58142571  0.03397752 -1.09660008]
 [ 2.5975882  0.22189396 -1.78521251 -1.61108875 -0.47427724 -0.42946978
 -0.19586628 -0.50134254 -0.68991325 -0.49506427  0.30992296  0.84277078
 -0.14340608 -0.14977735  0.40967899]
 [-0.84383581  1.47747917  0.68677158 -0.13136959  0.14453697  2.32174986
 -0.31895837 -1.88730148 -0.98844581 -0.55408945 -2.37047401  0.61261919
  0.82659297  2.13418249  0.75620133]
 [ 5.01708884  1.35266667 -0.43273835 -1.49660203 -0.23386624 -0.7187507
  0.08710456 -0.32073972 -0.38435889 -1.26764841 -0.26832647  1.34475712
 -0.53265453 -0.14577633 -0.26005533]]
```

SECTION 11: VISUALIZING DATA BEFORE AND AFTER PCA**CODE:**

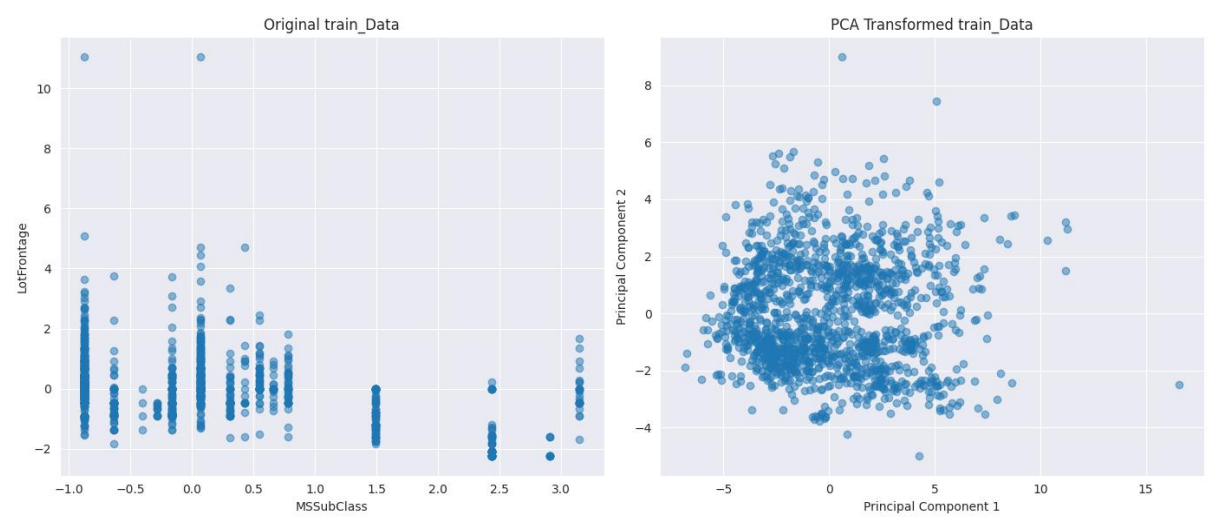
```
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
plt.scatter(train_data[numeric_features[0]],
            train_data[numeric_features[1]], alpha=0.5)
plt.title('Original train_Data')
plt.xlabel(numeric_features[0])
plt.ylabel(numeric_features[1])
```

CODE LINE	SIMPLE EXPLANATION
<code>plt.figure(figsize=(14, 6))</code>	Creates a figure for side-by-side plots.
<code>plt.subplot(1, 2, 1)</code>	Defines the first subplot.
<code>plt.scatter(...)</code>	Plots a scatter graph using two numeric features.
<code>plt.title('Original train_Data')</code>	Adds a title to the original data plot.
<code>plt.xlabel(...) / plt.ylabel(...)</code>	Labels the x- and y-axes using feature names.

CODE:

```
plt.subplot(1, 2, 2)
plt.scatter(train_data_pca[:, 0], train_data_pca[:, 1], alpha=0.5)
plt.title('PCA Transformed train_Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.tight_layout()
plt.show()
```

CODE LINE	SIMPLE EXPLANATION
<code>plt.subplot(1, 2, 2)</code>	Creates the second subplot in the figure.
<code>plt.scatter(train_data_pca[:, 0], train_data_pca[:, 1], alpha=0.5)</code>	Plots the first two principal components after PCA.
<code>plt.title('PCA Transformed train_Data')</code>	Adds a title to the PCA-transformed plot.
<code>plt.xlabel('Principal Component 1')</code>	Labels the x-axis as the first PCA component.
<code>plt.ylabel('Principal Component 2')</code>	Labels the y-axis as the second PCA component.
<code>plt.tight_layout()</code>	Adjusts spacing to avoid overlap.
<code>plt.show()</code>	Displays the plot.

OUTPUT:

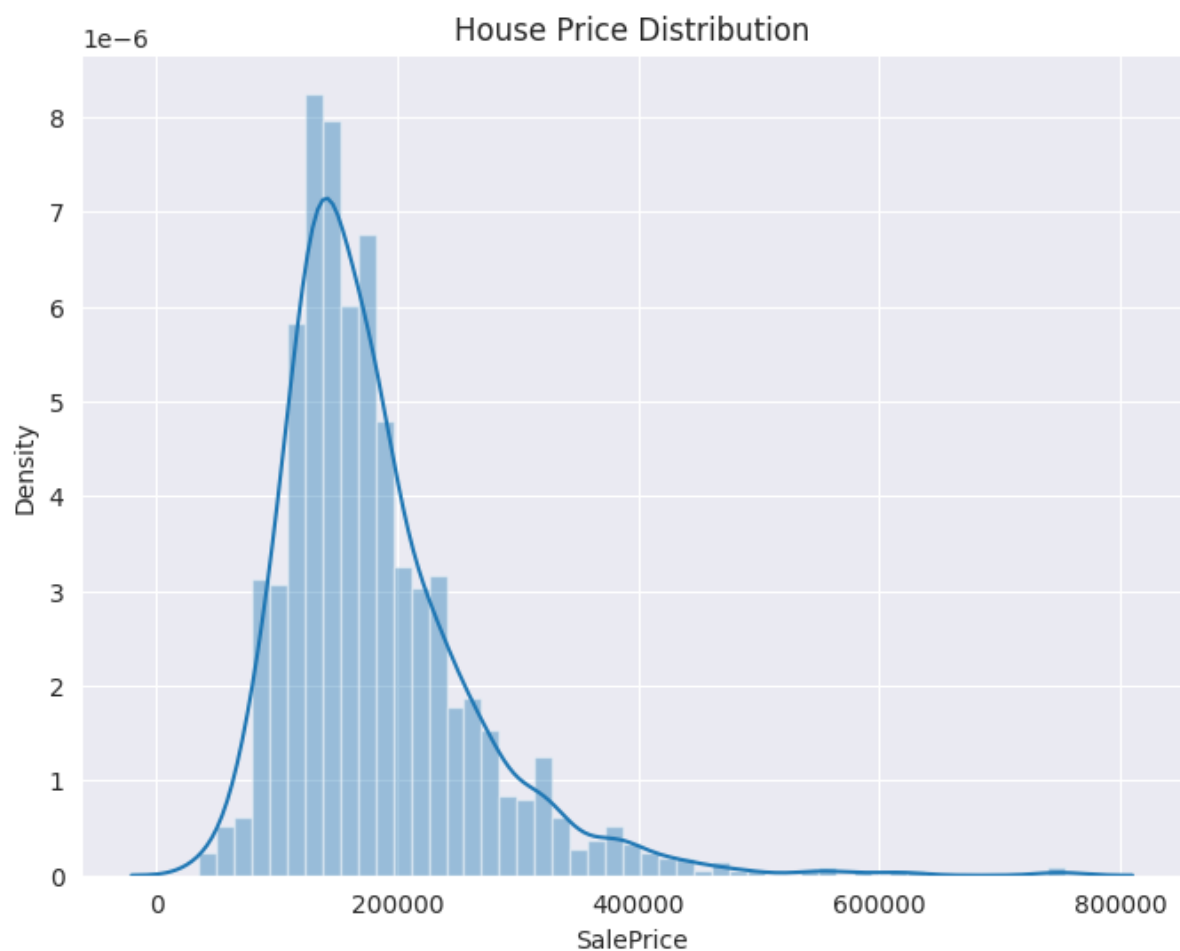
STEP 4

IMPLEMENTING LINEAR REGRESSION, MULTIPLE LINEAR REGRESSION

CODE:

```
plt.figure(figsize=(8,6))
sns.distplot(y)
title = plt.title("House Price Distribution")
```

OUTPUT:



CODE:

```

from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression

estimator = LinearRegression()
rfe = RFE(estimator, n_features_to_select=10, step=1)
selector = rfe.fit(X_train.fillna(0).select_dtypes(exclude='object'), y)
selectedFeatures = list(
    X.select_dtypes(exclude='object').columns[selector.support_])
selectedFeatures

```

OUTPUT:

```

['OverallQual',
 'BsmtFullBath',
 'BsmtHalfBath',
 'FullBath',
 'HalfBath',
 'BedroomAbvGr',
 'KitchenAbvGr',
 'TotRmsAbvGrd',
 'Fireplaces',
 'GarageCars']

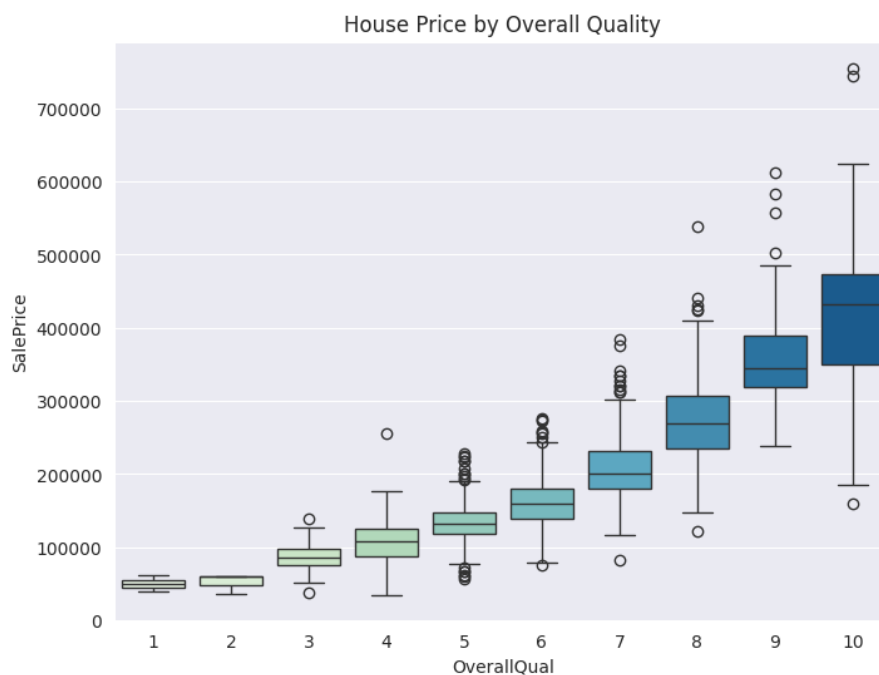
```

CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
RFE and LinearRegression	Imports Recursive Feature Elimination and Linear Regression.
estimator = LinearRegression()	Creates a linear regression model.
RFE(estimator, n_features_to_select=10)	Selects top 10 important features.
X_train.fillna(0).select_dtypes(exclude='object')	Removes NaNs and selects numerical features.
selector.support_	Boolean mask for selected features.
list(...columns[selector.support_])	Lists selected feature names.

CODE:

```
plt.figure(figsize=(8, 6))
sns.boxplot(x='OverallQual', y='SalePrice', data=train_data, palette='GnBu')
title = plt.title('House Price by Overall Quality')
```

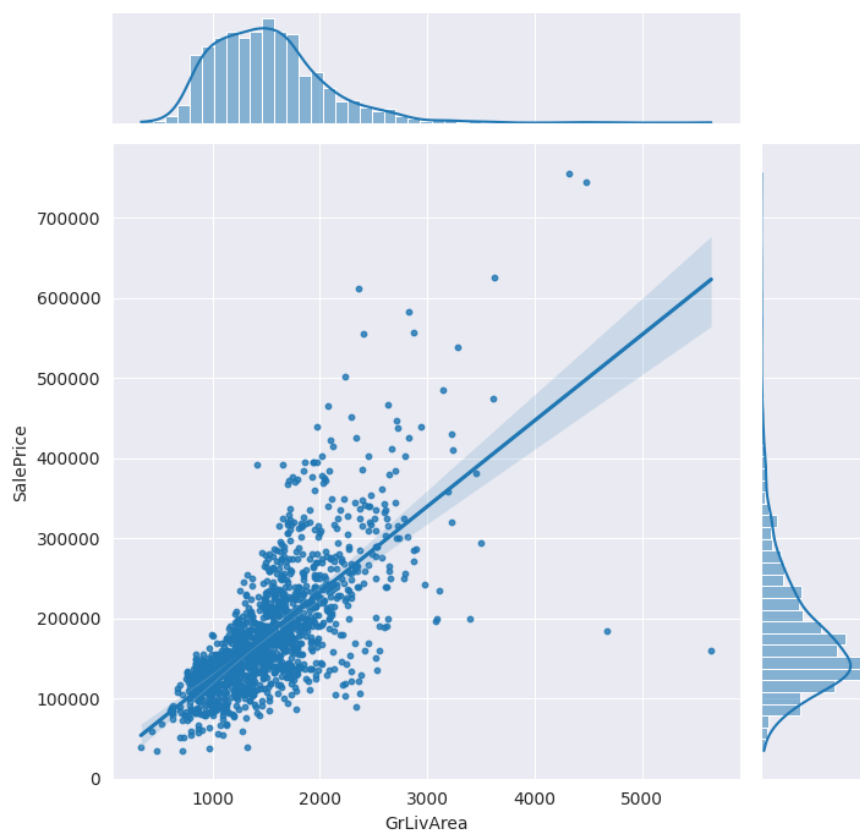
OUTPUT:**CODE EXPLANATION:**

CODE LINE	SIMPLE EXPLANATION
<code>plt.figure(figsize=(8, 6))</code>	Sets figure size.
<code>sns.boxplot(...)</code>	Plots boxplot of SalePrice vs Overall Quality.
<code>plt.title(...)</code>	Adds title to the plot.

CODE:

```
def plotCorrelation(variables):  
    """  
    1. Print correlation of two variables  
    2. Create jointplot of two variables  
    """  
    # Print correlation  
    print("Correlation: ", train_data[[variables[0], variables[1]]].corr().iloc[1,  
0])  
  
    # Create jointplot  
    sns.jointplot(  
        x=variables[0],  
        y=variables[1],  
        data=train_data,  
        kind='reg',  
        height=7,  
        scatter_kws={'s': 10},  
        marginal_kws={'kde': True}  
    )  
  
    # Example usage  
    plotCorrelation(['GrLivArea', 'SalePrice'])
```

OUTPUT:

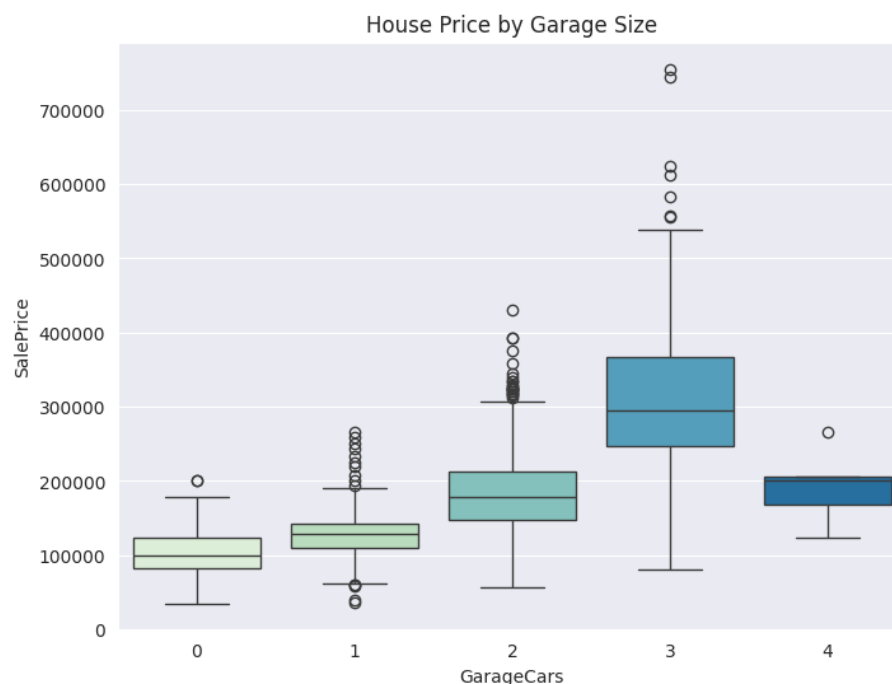


CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
<code>def plotCorrelation(...)</code>	Defines a function to check and plot correlation.
<code>corr().iloc[1, 0]</code>	Calculates correlation coefficient between 2 variables.
<code>sns.jointplot(...)</code>	Plots scatter + regression + KDE for both variables.

CODE:

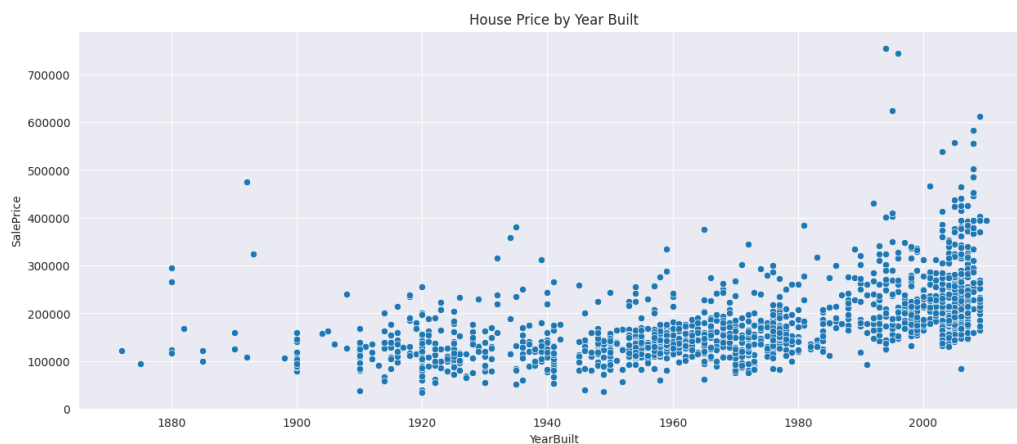
```
plt.figure(figsize=(8, 6))
sns.boxplot(x='GarageCars', y='SalePrice', data=train_data, palette='GnBu')
title = plt.title('House Price by Garage Size')
```

OUTPUT:**CODE EXPLANATION:**

CODE LINE	SIMPLE EXPLANATION
<code>sns.boxplot(...)</code>	Compares sale price across different garage sizes.

CODE:

```
plt.figure(figsize=(15, 6))
sns.scatterplot(x='YearBuilt', y='SalePrice', data=train_data)
title = plt.title('House Price by Year Built')
```

OUTPUT:**CODE EXPLANATION:**

CODE LINE	SIMPLE EXPLANATION
<code>sns.scatterplot(...)</code>	Shows how house price changes with year built.

CODE:

```
sigCatCols = [
    'Street', 'LandContour', 'LotConfig', 'LandSlope', 'Neighborhood',
    'Condition1', 'Condition2', 'RoofMatl', 'ExterQual', 'BsmtQual',
    'BsmtExposure', 'KitchenQual', 'Functional', 'GarageQual', 'PoolQC'
]
```

CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
<code>sigCatCols = [...]</code>	Stores selected categorical features to explore.

CODE:

```
def visualizeCatFeature(feature):
    """
    Visualize the relationship between 'SalePrice' and categorical feature using box
    plots
    """
    # Descending order of levels sorted by median SalePrice
    featOrder = train_data.groupby(
        [feature]).median().SalePrice.sort_values(ascending=False).index

    # Create box plot
    sns.boxplot(x=feature,
                y='SalePrice',
                data=train_data,
                order=featOrder,
                palette='GnBu_r')
```

CODE EXPLANATION:

CODE LINE	SIMPLE EXPLANATION
visualizeCatFeature(feature)	Defines function to plot boxplot for a categorical feature.
groupby(...).median().sort_values()	Sorts feature levels by median SalePrice.
sns.boxplot(...)	Creates a boxplot for the selected feature.

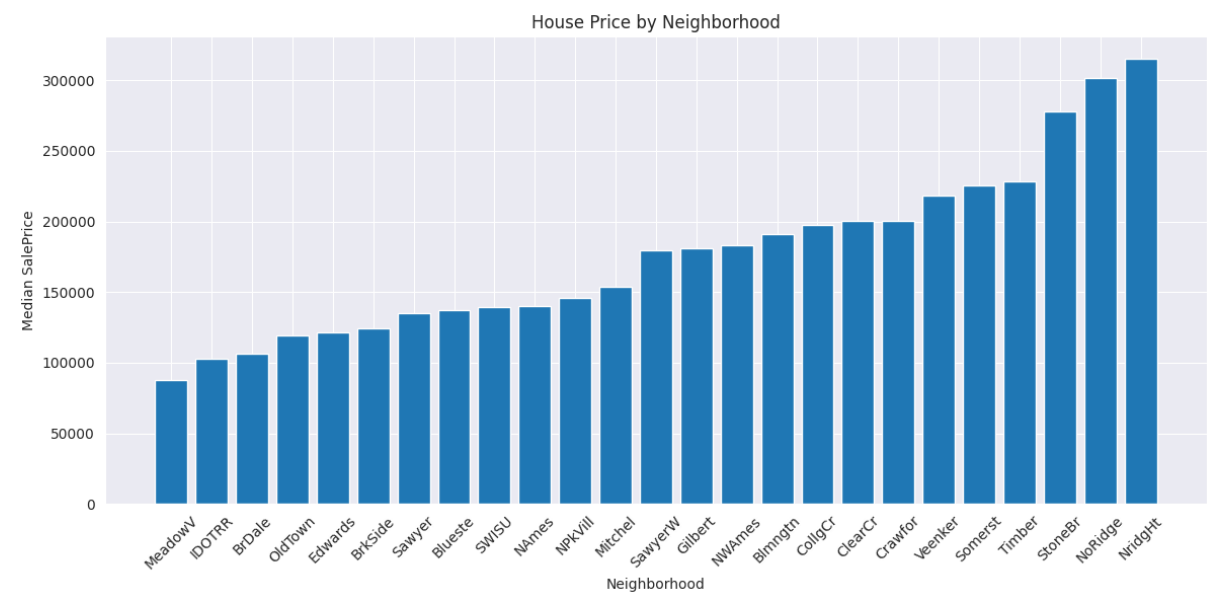
CODE:

```
def visualizeCatFeature(col):
    """
    Visualize the median SalePrice for each category in the specified column.
    """
    cat_price = train_data.groupby(col)['SalePrice'].median().sort_values()

    plt.bar(cat_price.index, cat_price.values)
    plt.xlabel(col)
    plt.ylabel('Median SalePrice')
```

CODE:

```
plt.figure(figsize=(12, 6))
visualizeCatFeature('Neighborhood')
plt.title('House Price by Neighborhood')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:**SUMMARY OF OUTPUT:**

This step performs feature selection using Recursive Feature Elimination (RFE) and explores relationships between different features and the target variable using boxplots, scatter plots, and correlation analysis. These steps help in identifying important features and understanding data distribution patterns visually.

STEP 5

LOGISTIC REGRESSION FOR BINARY CLASSIFICATION & MODEL EVALUATION USING METRICS (R^2 , MAE, MSE)

Logistic Regression is a supervised machine learning algorithm used for binary classification tasks—problems where the target variable has two possible outcomes (like "Yes" or "No"). It models the probability of a certain class or event. Model evaluation metrics help us understand how well our regression or classification models are performing. Common metrics include R^2 (coefficient of determination), MAE (Mean Absolute Error), and MSE (Mean Squared Error).

HANDLING MISSING VALUES (MULTIPLE STRATEGIES)

CODE:

```
from sklearn.impute import SimpleImputer
group_1 = [
    'PoolQC', 'MiscFeature', 'Alley', 'Fence', 'FireplaceQu', 'GarageType',
    'GarageFinish', 'GarageQual', 'GarageCond', 'BsmtQual', 'BsmtCond',
    'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2', 'MasVnrType'
]
X[group_1] = X[group_1].fillna("None")
```

CODE LINE	SIMPLE EXPLANATION
from sklearn.impute import SimpleImputer	Imports a tool to fill missing values with a selected strategy.
group_1 = [...]	Defines a group of categorical columns for imputation.
X[group_1] = X[group_1].fillna("None")	Fills missing values in group_1 columns with "None".

CODE:

```

group_2 = [
    'GarageArea', 'GarageCars', 'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF',
    'TotalBsmtSF', 'BsmtFullBath', 'BsmtHalfBath', 'MasVnrArea'
]
X[group_2] = X[group_2].fillna(0)

```

CODE LINE	SIMPLE EXPLANATION
group_2 = [...]	Lists numeric features to be filled with 0.
X[group_2] = X[group_2].fillna(0)	Fills missing values in group_2 with zeros.

CODE:

```

group_3a = [
    'Functional', 'MSZoning', 'Electrical', 'KitchenQual', 'Exterior1st',
    'Exterior2nd', 'SaleType', 'Utilities'
]
imputer = SimpleImputer(strategy='most_frequent')
X[group_3a] = pd.DataFrame(imputer.fit_transform(X[group_3a]),
index=X.index)

```

CODE LINE	SIMPLE EXPLANATION
group_3a = [...]	Lists categorical features for mode-based imputation.
imputer = SimpleImputer(strategy='most_frequent')	Sets up imputer to replace missing values with the most frequent one.
X[group_3a] pd.DataFrame(imputer.fit_transform(X[group_3a]), index=X.index)	= Applies the imputer and restores DataFrame structure.

CODE:

```

X.LotFrontage = X.LotFrontage.fillna(X.LotFrontage.mean())
X.GarageYrBlt = X.GarageYrBlt.fillna(X.YearBuilt)
sum(X.isnull().sum())

```

CODE LINE	SIMPLE EXPLANATION
X.LotFrontage = X.LotFrontage.fillna(X.LotFrontage.mean())	Replaces missing values with column mean.
X.GarageYrBlt = X.GarageYrBlt.fillna(X.YearBuilt)	Replaces missing garage year with house build year.
sum(X.isnull().sum())	Computes total number of remaining missing values.

VISUALIZING AND REMOVING OUTLIERS

CODE:

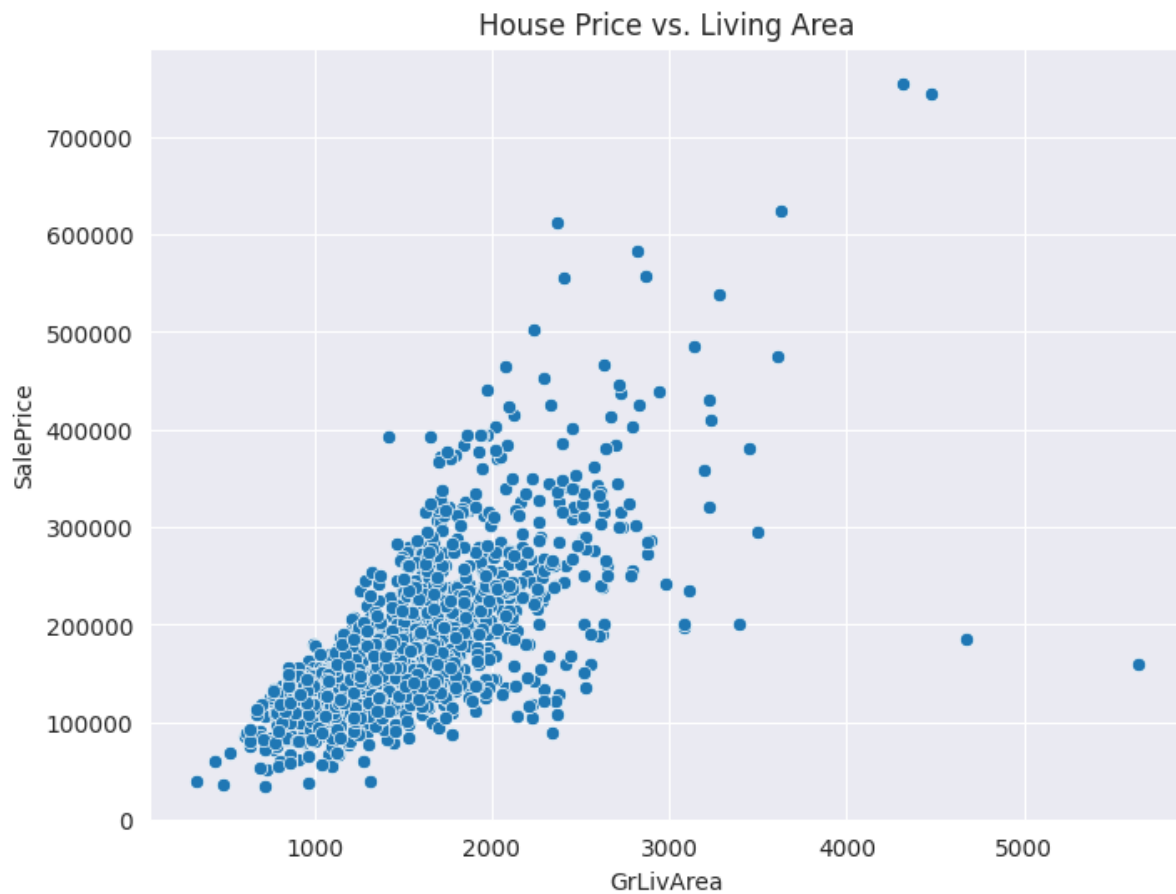
```

sns.set_style('darkgrid')
plt.figure(figsize=(8, 6))
sns.scatterplot(x='GrLivArea', y='SalePrice', data=train_data)
title = plt.title('House Price vs. Living Area')

```

CODE LINE	SIMPLE EXPLANATION
sns.set_style('darkgrid')	Sets the seaborn plot style to dark grid for visual clarity.
plt.figure(figsize=(8, 6))	Creates a plot figure with specified width and height.
sns.scatterplot(x='GrLivArea', y='SalePrice', data=train_data)	Draws a scatter plot of living area vs. house price.
title = plt.title('House Price vs. Living Area')	Adds a title to the plot.

OUTPUT:



CODE:

```
outlier_index = train_data[(train_data.GrLivArea > 4000) &
(train_data.SalePrice < 200000)].index
X.drop(outlier_index, axis=0, inplace=True)
y.drop(outlier_index, axis=0, inplace=True)
```

CODE LINE	SIMPLE EXPLANATION	
outlier_index = ...	Identifies outliers: homes with very large area but low price.	
X.drop(outlier_index, axis=0, inplace=True)	axis=0,	Removes those outlier rows from input features.
y.drop(outlier_index, inplace=True)	axis=0,	Removes the corresponding target values.

FEATURE ENGINEERING

CODE:

```
X['totalSqFeet'] = X['TotalBsmtSF'] + X['1stFlrSF'] + X['2ndFlrSF']
```

CODE LINE	SIMPLE EXPLANATION
X['totalSqFeet'] = X['TotalBsmtSF'] + X['1stFlrSF'] + X['2ndFlrSF']	Creates a new feature that combines all floor areas into total square feet.

CODE:

```
X['totalBathroom'] = X.FullBath + X.BsmtFullBath + 0.5 * (X.HalfBath + X.BsmtHalfBath)
```

CODE LINE	SIMPLE EXPLANATION
X['totalBathroom'] = ...	Creates a new feature combining full and half baths (half baths count as 0.5).

CODE:

```
X['houseAge'] = X.YrSold - X.YearBuilt
X['reModeled'] = np.where(X.YearRemodAdd == X.YearBuilt, 0, 1)
X['isNew'] = np.where(X.YrSold == X.YearBuilt, 1, 0)
```

CODE LINE	SIMPLE EXPLANATION
X['houseAge'] = X.YrSold - X.YearBuilt	Calculates the age of the house at the time it was sold.

<code>X['reModeled'] = np.where(X.YearRemodAdd == X.YearBuilt, 0, 1)</code>	Indicates if the house was remodeled (1) or not (0).
<code>X['isNew'] = np.where(X.YrSold == X.YearBuilt, 1, 0)</code>	Indicates if the house was newly built (1) or not (0).

ENCODING CATEGORICAL VARIABLES

CODE:

```
label_encoding_cols = [
    "Alley", "BsmtCond", "BsmtExposure", "BsmtFinType1", "BsmtFinType2",
    "BsmtQual", "ExterCond", "ExterQual", "FireplaceQu", "Functional",
    "GarageCond", "GarageQual", "HeatingQC", "KitchenQual", "LandSlope",
    "LotShape", "PavedDrive", "PoolQC", "Street", "Utilities"
]
label_encoder = LabelEncoder()
for col in label_encoding_cols:
    X[col] = label_encoder.fit_transform(X[col])
```

CODE LINE	SIMPLE EXPLANATION
<code>label_encoding_cols = [...]</code>	Lists categorical features to encode numerically.
<code>label_encoder = LabelEncoder()</code>	Initializes the label encoder.
<code>for col in label_encoding_cols: X[col] = label_encoder.fit_transform(X[col])</code>	Encodes each listed column with numeric labels.

CODE:

```
to_factor_cols = ['YrSold', 'MoSold', 'MSSubClass']
for col in to_factor_cols:
    X[col] = X[col].apply(str)
```

CODE LINE	SIMPLE EXPLANATION
<code>to_factor_cols = ['YrSold', 'MoSold', 'MSSubClass']</code>	Defines columns to be converted into categorical (string) types.

for col in to_factor_cols: X[col] = X[col].apply(str)	Converts the values of these columns into strings.
--	--

TARGET TRANSFORMATION & SKEWNESS CORRECTION

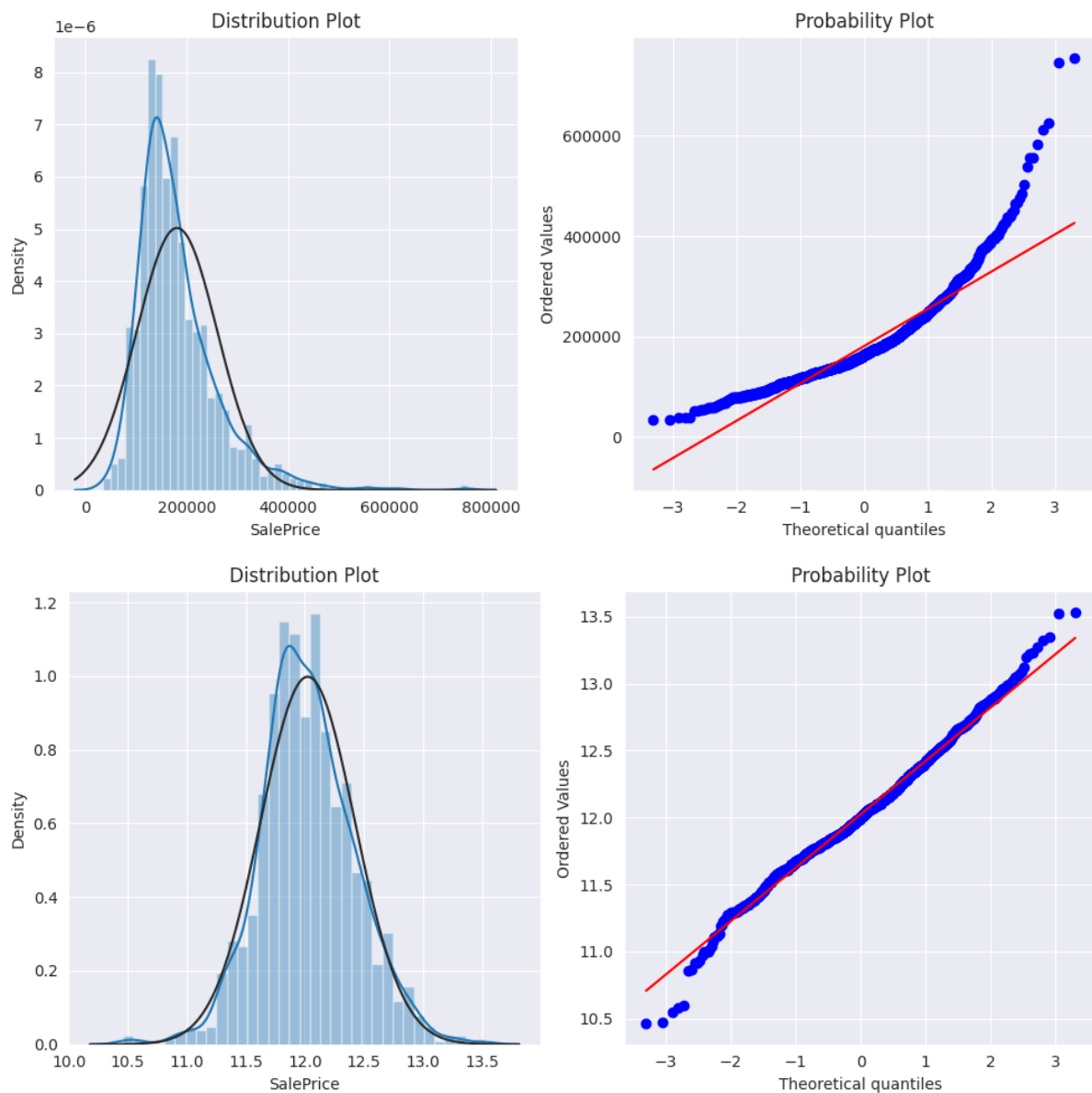
CODE:

```
from scipy.stats import norm
def normality_plot(X):
    fig, axes = plt.subplots(1, 2, figsize=(10, 5))
    sns.distplot(X, fit=norm, ax=axes[0])
    axes[0].set_title('Distribution Plot')
    axes[1] = stats.probplot(X, plot=plt)
    plt.tight_layout()
```

CODE LINE	SIMPLE EXPLANATION
from scipy.stats import norm	Imports the normal distribution object for fitting.
def normality_plot(X): ...	Defines a function to plot histogram and Q-Q plot for a variable to check normality.

```
normality_plot(y)
y = np.log(1 + y)
normality_plot(y)
```

CODE LINE	SIMPLE EXPLANATION
normality_plot(y)	Plots the distribution of the original target variable y.
y = np.log(1 + y)	Applies a log transformation to reduce skewness in the target variable.
normality_plot(y)	Plots the distribution of the transformed target variable.

OUTPUT:**CODE:**

```
numeric_data = train_data.select_dtypes(include='number')
skewness = numeric_data.skew().sort_values(ascending=False)
skewness[abs(skewness) > 0.75]
```

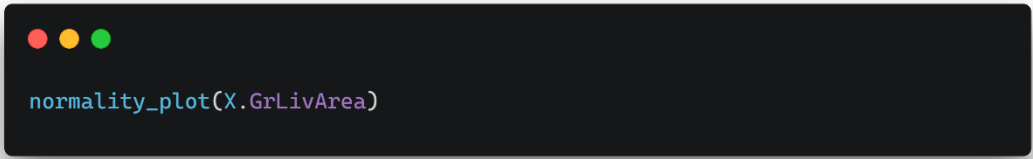
CODE LINE		SIMPLE EXPLANATION
numeric_data	=	Selects all numeric columns from the training dataset.
train_data.select_dtypes(include='number')		
skewness	=	Computes and sorts the skewness of numeric features.
numeric_data.skew().sort_values(ascending=False)		
skewness[abs(skewness) > 0.75]		Filters features with high skewness (absolute > 0.75).

OUTPUT:

```

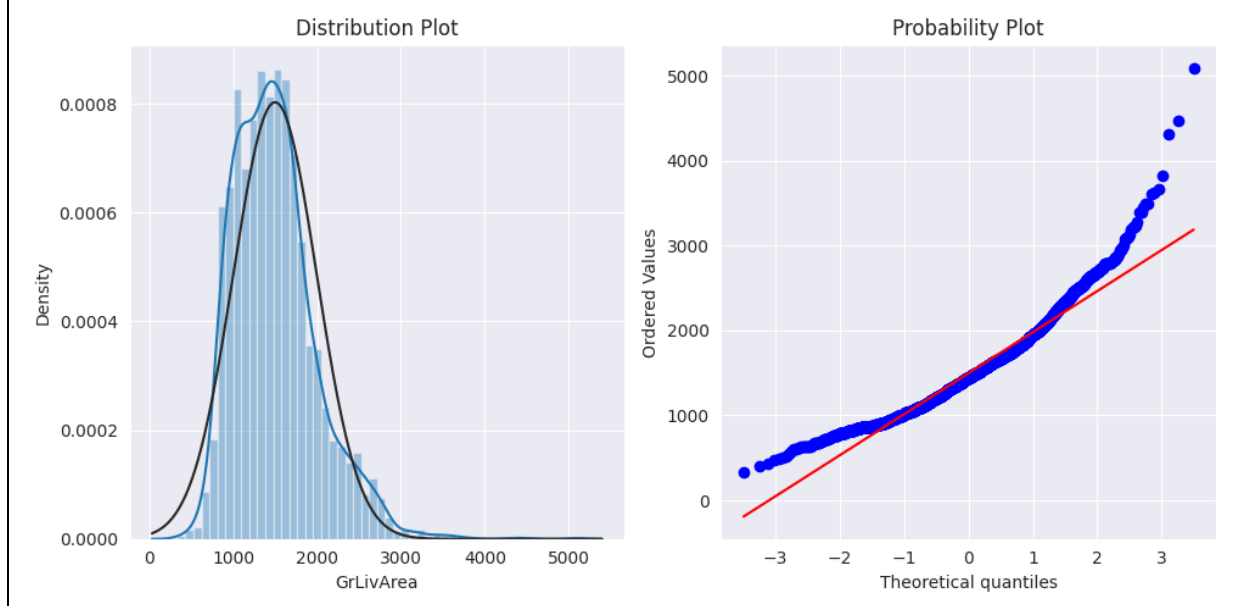
MiscVal      24.476794
PoolArea     14.828374
LotArea      12.207688
3SsnPorch    10.304342
LowQualFinSF  9.011341
KitchenAbvGr  4.488397
BsmtFinSF2    4.255261
ScreenPorch   4.122214
BsmtHalfBath  4.103403
EnclosedPorch 3.089872
MasVnrArea    2.669084
OpenPorchSF   2.364342
LotFrontage   2.163569
SalePrice     1.882876
BsmtFinSF1    1.685503
WoodDeckSF    1.541376
TotalBsmtSF   1.524255
MSSubClass    1.407657
1stFlrSF      1.376757
GrLivArea     1.366560
BsmtUnfSF     0.920268
2ndFlrSF      0.813030
dtype: float64

```

CODE:


```
normality_plot(X.GrLivArea)
```

CODE LINE	SIMPLE EXPLANATION
normality_plot(X.GrLivArea)	Plots the distribution of the GrLivArea feature.

OUTPUT:**CODE:**

```
skewed_cols = list(skewness[abs(skewness) > 0.5].index)
skewed_cols = [col for col in skewed_cols if col not in ['MSSubClass',
'SalePrice']]
for col in skewed_cols:
    X[col] = np.log(1 + X[col])
```

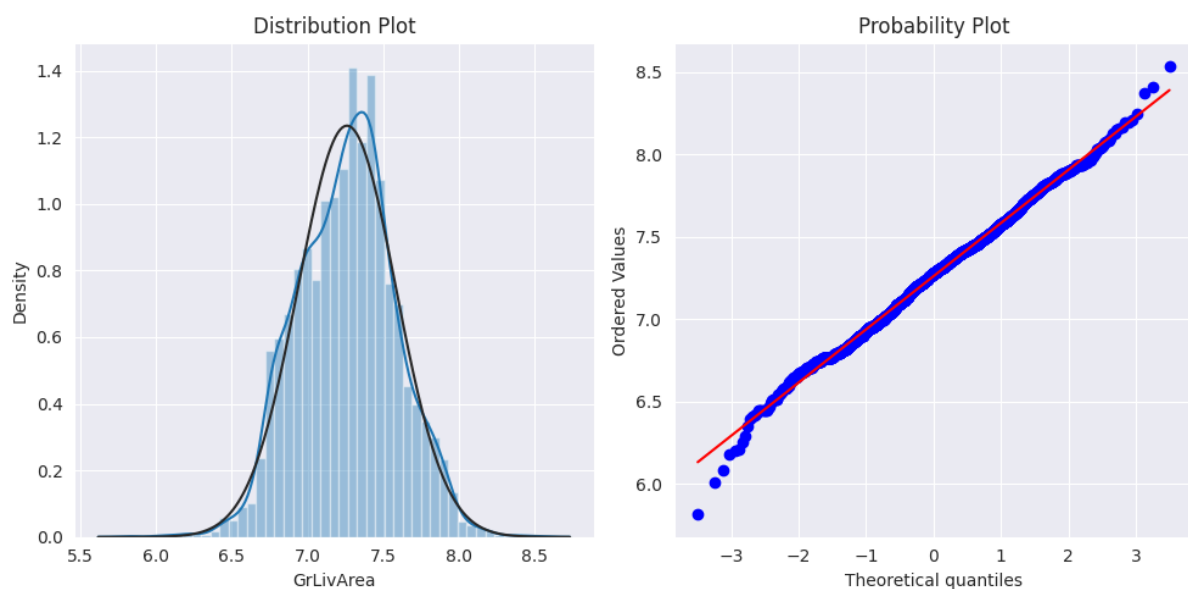
CODE LINE	SIMPLE EXPLANATION
<code>skewed_cols = list(skewness[abs(skewness) > 0.5].index)</code>	Lists all columns with moderate to high skewness.
<code>skewed_cols = [col for col in skewed_cols if col not in ['MSSubClass', 'SalePrice']]</code>	Excludes specific columns from transformation.
<code>for col in skewed_cols: X[col] = np.log(1 + X[col])</code>	Applies $\log(1 + x)$ transformation to each skewed column.

CODE:

```
normality_plot(X.GrLivArea)
```

CODE LINE	SIMPLE EXPLANATION
<code>normality_plot(X.GrLivArea)</code>	Plots the transformed GrLivArea distribution to confirm normality.

OUTPUT:



SCALING NUMERIC FEATURES & ENCODING CATEGORICALS

CODE:

```
from sklearn.preprocessing import RobustScaler
numerical_cols = list(X.select_dtypes(exclude=['object']).columns)
scaler = RobustScaler()
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])
```

CODE LINE	SIMPLE EXPLANATION
from sklearn.preprocessing import RobustScaler	Imports a scaler that's robust to outliers.
numerical_cols = list(X.select_dtypes(exclude=['object']).columns)	Selects all numeric columns for scaling.
scaler = RobustScaler()	Initializes the RobustScaler.
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])	Applies robust scaling to all numeric features.

CODE:

```
X = pd.get_dummies(X, drop_first=True)
print("X.shape:", X.shape)
```

CODE LINE	SIMPLE EXPLANATION
X = pd.get_dummies(X, drop_first=True)	Converts categorical columns into dummy/indicator variables (one-hot encoding) while avoiding multicollinearity.
print("X.shape:", X.shape)	Prints the shape of the DataFrame after encoding.

OUTPUT:

X.SHAPE: (2917, 237)

SPLITTING DATA & SETTING UP CROSS-VALIDATION

CODE:

```
n_test = len(test_data)
X_train = X.iloc[:-n_test, :]
X_test = X.iloc[-n_test:, :]
print("X_train.shape:", X_train.shape)
print("X_test.shape:", X_test.shape)
```


CODE LINE	SIMPLE EXPLANATION
<code>ntest = len(test_data)</code>	Stores the number of rows in the test dataset.
<code>X_train = X.iloc[:ntest, :]</code>	Selects the top portion of the data for training (excluding test set).
<code>X_test = X.iloc[-ntest:, :]</code>	Selects the bottom portion of the data for testing.
<code>print("X_train.shape:", X_train.shape)</code>	Prints the shape of the training feature set.
<code>print("X_test.shape:", X_test.shape)</code>	Prints the shape of the test feature set.

OUTPUT:

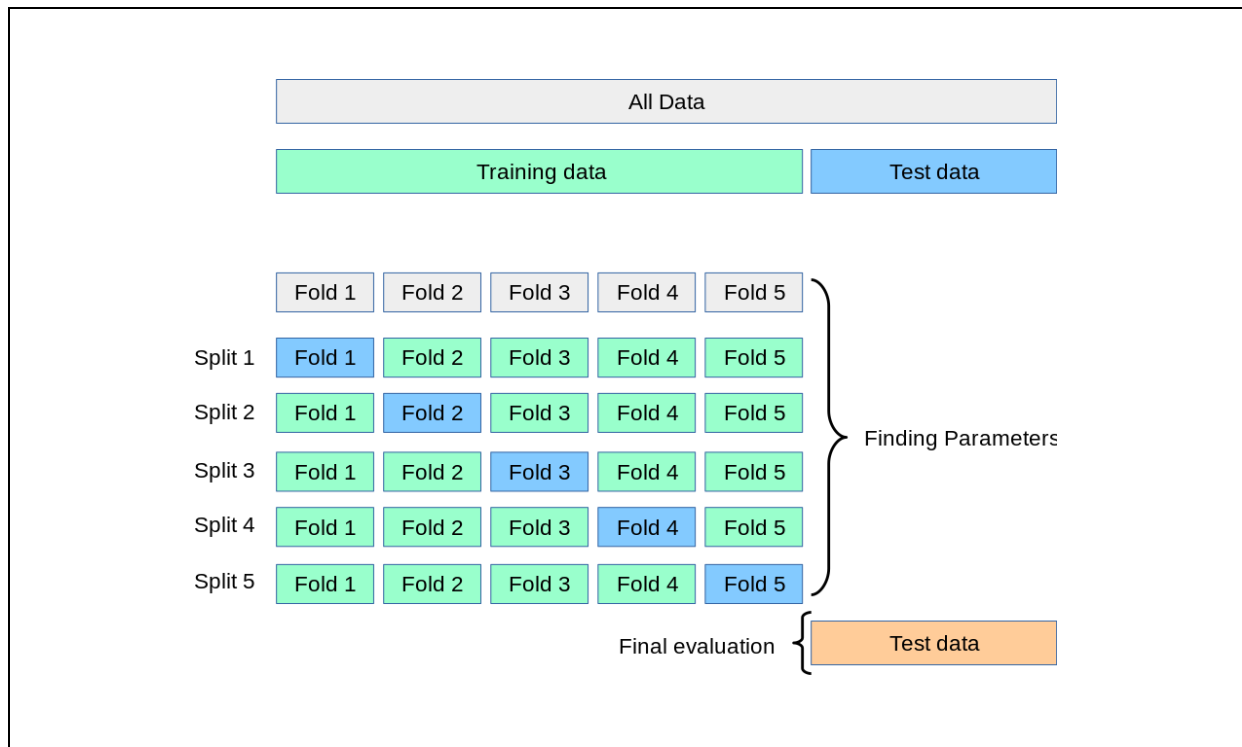
`X_train.shape: (1458, 237)`

`X_test.shape: (1459, 237)`

CODE:

```
from sklearn.model_selection import KFold, cross_val_score  
n_folds = 5
```

CODE LINE	SIMPLE EXPLANATION
<code>from sklearn.model_selection import KFold, cross_val_score</code>	Imports cross-validation utilities from scikit-learn.
<code>n_folds = 5</code>	Sets the number of folds for K-Fold cross-validation.

**CODE:**

```
def getRMSLE(model):
    kf = KFold(n_folds, shuffle=True, random_state=42)
    rmse = np.sqrt(-cross_val_score(
        model, X_train, y, scoring="neg_mean_squared_error", cv=kf))
    return rmse.mean()
```

CODE LINE	SIMPLE EXPLANATION
def getRMSLE(model):	Defines a function to compute Root Mean Squared Log Error (RMSLE).
kf = KFold(n_folds, shuffle=True, random_state=42)	Initializes the K-Fold strategy with reproducible shuffling.
rmse = np.sqrt(-cross_val_score(...))	Calculates cross-validated RMSE (square root of negative MSE).
return rmse.mean()	Returns the mean RMSE across folds.

RIDGE AND LASSO REGRESSION WITH HYPERPARAMETER TUNING

CODE:

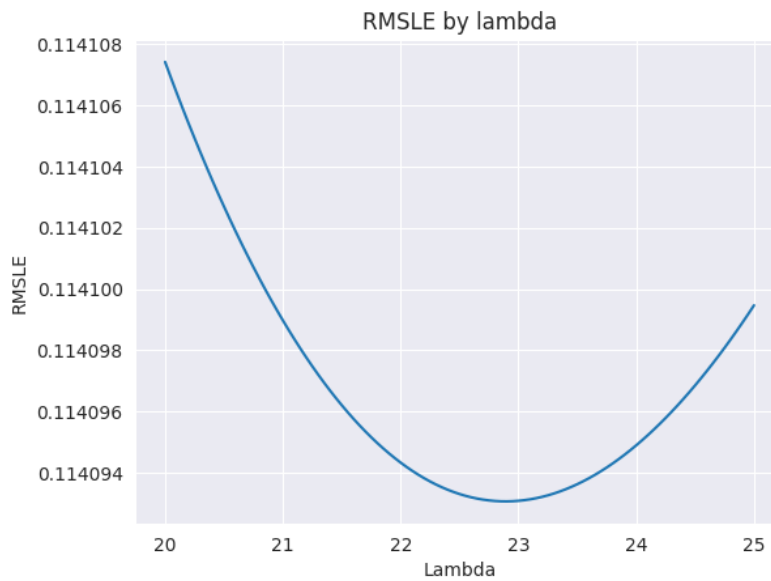
```
from sklearn.linear_model import Ridge, Lasso
lambda_list = list(np.linspace(20, 25, 101))
rmsle_ridge = [getRMSLE(Ridge(alpha=lambda_)) for lambda_ in lambda_list]
rmsle_ridge = pd.Series(rmsle_ridge, index=lambda_list)
```

CODE LINE	SIMPLE EXPLANATION
from sklearn.linear_model import Ridge, Lasso	Imports Ridge and Lasso regression models from scikit-learn.
lambda_list = list(np.linspace(20, 25, 101))	Generates 101 evenly spaced values between 20 and 25 (as regularization parameters).
rmsle_ridge = [getRMSLE(Ridge(alpha=lambda_)) for lambda_ in lambda_list]	Computes RMSLE for each Ridge model with a different alpha (λ).
rmsle_ridge = pd.Series(rmsle_ridge, index=lambda_list)	Converts results into a Pandas Series indexed by lambda values.

CODE:

```
rmsle_ridge.plot(title="RMSLE by lambda")
plt.xlabel("Lambda")
plt.ylabel("RMSLE")
```

CODE LINE	SIMPLE EXPLANATION
rmsle_ridge.plot(title="RMSLE by lambda")	Plots RMSLE values across all lambda values for Ridge regression.
plt.xlabel("Lambda")	Labels the x-axis as lambda.
plt.ylabel("RMSLE")	Labels the y-axis as RMSLE.

OUTPUT:**CODE:**

```
print("Best lambda:", rmsle_ridge.idxmin())
print("RMSLE:", rmsle_ridge.min())
ridge = Ridge(alpha=22.9)
```

CODE LINE	SIMPLE EXPLANATION
print("Best lambda:", rmsle_ridge.idxmin())	Prints the lambda value that gave the lowest RMSLE.
print("RMSLE:", rmsle_ridge.min())	Prints the corresponding lowest RMSLE value.
ridge = Ridge(alpha=22.9)	Initializes the final Ridge model using the optimal lambda value.

CODE:

```

lambda_list = list(np.linspace(0.0006, 0.0007, 11))
rmsle_lasso = [getRMSLE(Lasso(alpha=lambda_, max_iter=100000)) for lambda_
in lambda_list]
rmsle_lasso = pd.Series(rmsle_lasso, index=lambda_list)

```

CODE LINE	SIMPLE EXPLANATION
lambda_list = list(np.linspace(0.0006, 0.0007, 11))	Generates 11 lambda values between 0.0006 and 0.0007 for Lasso.
rmsle_lasso = [getRMSLE(Lasso(alpha=lambda_, max_iter=100000)) for lambda_ in lambda_list]	Evaluates each Lasso model using cross-validated RMSLE.
rmsle_lasso = pd.Series(rmsle_lasso, index=lambda_list)	Stores RMSLE results in a Series indexed by lambda.

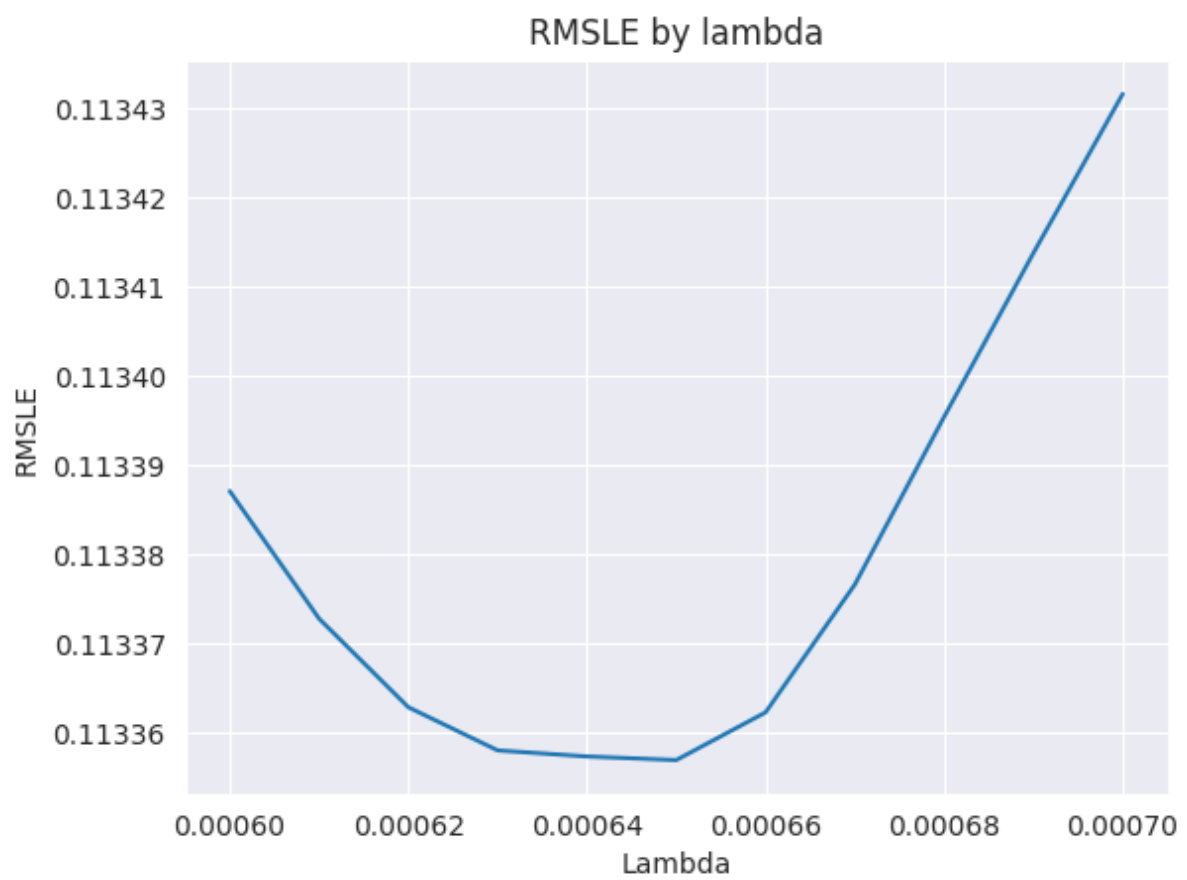
CODE:

```

rmsle_lasso.plot(title="RMSLE by lambda")
plt.xlabel("Lambda")
plt.ylabel("RMSLE")
print("Best lambda:", rmsle_lasso.idxmin())
print("RMSLE:", rmsle_lasso.min())
lasso = Lasso(alpha=0.00065, max_iter=100000)

```

CODE LINE	SIMPLE EXPLANATION
rmsle_lasso.plot(title="RMSLE by lambda")	Plots RMSLE values for each lambda used in Lasso regression.
plt.xlabel("Lambda")	Sets the label for the x-axis.
plt.ylabel("RMSLE")	Sets the label for the y-axis.
print("Best lambda:", rmsle_lasso.idxmin())	Displays the lambda value with the best performance.
print("RMSLE:", rmsle_lasso.min())	Shows the lowest RMSLE score.
lasso = Lasso(alpha=0.00065, max_iter=100000)	Initializes the final Lasso model with the chosen lambda.

OUTPUT:

XGBOOST AND LIGHTGBM REGRESSION MODELS

CODE:

```
from xgboost import XGBRegressor
xgb = XGBRegressor(
    learning_rate=0.05,
    n_estimators=2100,
    max_depth=2,
    min_child_weight=2,
    gamma=0,
    subsample=0.65,
    colsample_bytree=0.46,
    scale_pos_weight=1,
    reg_alpha=0.464,
    reg_lambda=0.8571,
    random_state=7,
    n_jobs=2,
    verbosity=0
)
getRMSLE(xgb)
```

CODE LINE	SIMPLE EXPLANATION
from xgboost import XGBRegressor	Imports the XGBoost regressor model.
xgb = XGBRegressor(...)	Initializes the XGBoost model with specific hyperparameters.
getRMSLE(xgb)	Evaluates XGBoost model performance using RMSLE.

CODE:

```

from lightgbm import LGBMRegressor
X_train.columns = X_train.columns.str.replace(' ', '_')
lgb = LGBMRegressor(
    objective='regression',
    learning_rate=0.05,
    n_estimators=730,
    num_leaves=8,
    min_data_in_leaf=4,
    max_depth=3,
    max_bin=55,
    bagging_fraction=0.78,
    bagging_freq=5,
    feature_fraction=0.24,
    feature_fraction_seed=9,
    bagging_seed=9,
    min_sum_hessian_in_leaf=11,
    verbosity=-1
)
getRMSLE(lgb)

```

CODE LINE	SIMPLE EXPLANATION
from lightgbm import LGBMRegressor	Imports the LightGBM regressor model.
X_train.columns = X_train.columns.str.replace(' ', '_')	Replaces spaces in column names with underscores to avoid issues.
lgb = LGBMRegressor(...)	Initializes the LightGBM model with tuning parameters.
getRMSLE(lgb)	Evaluates LightGBM model performance using RMSLE.

OUTPUT:

[Space for student to write/paste RMSLE result]

SUMMARY OF OUTPUT:

- Missing values are handled using different strategies depending on the column type.
- Outliers are removed and new features (like total square feet, total bathrooms, etc.) are added.
- Features with high skewness are log-transformed for normality.
- Data is robustly scaled and categorical variables are one-hot encoded.
- The dataset is split into training and testing parts, with their shapes printed.
- Multiple regression models are evaluated using RMSLE (a common regression metric).
- The best-performing regression models are combined using an averaging approach.
- Final predictions are made and saved for submission.

PROJECT NO 2: CLASSIFICATION

PROJECT NO 2: CUSTOMER CHURN PREDICTION

This project is an **Exploratory Data Analysis (EDA)** for a **customer churn prediction** model. It analyzes telecom customer data to identify patterns and build predictive models for customer attrition ("Churn"). Key steps include data preprocessing, visualization, feature engineering, model training/evaluation, and hyperparameter tuning using machine learning and deep learning techniques.

KEY STEPS & WORKFLOW

DATA LOADING & CLEANING:

- Load churn.csv (7,043 rows, 21 features).
- Clean column names (strip whitespace).
- Check for missing values (none found).

EXPLORATORY ANALYSIS:

- Visualize target variable (Churn) distribution using Plotly (pie chart).
- Analyze feature categories: demographics, services, account details.

PREPROCESSING:

- Encode categorical variables (e.g., gender, Contract).
- Scale numerical features (tenure, MonthlyCharges, TotalCharges).
- Split data into train/test sets.

MODELING:

- Train **5 classifiers**:
 - Logistic Regression
 - Random Forest
 - Decision Tree
 - Gaussian Naive Bayes
 - SVM
- Build a **neural network** (Keras Sequential model).
- Optimize models via RandomizedSearchCV (Random Forest).

EVALUATION:

- Metrics: Accuracy, Precision, Recall, F1-score.
- Confusion matrices.
- Save best model (joblib).

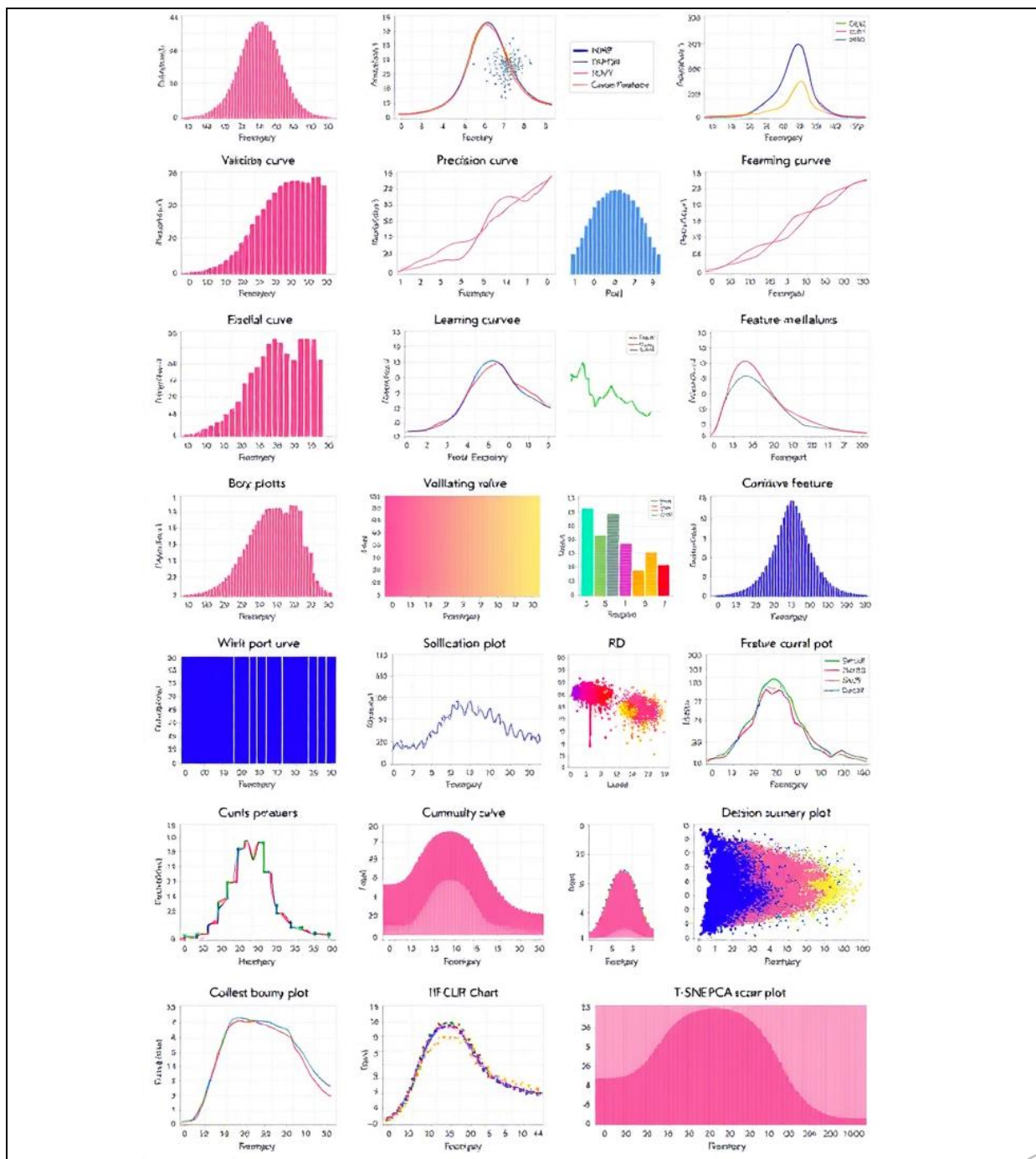
LIBRARIES & TOOLS

- **Data Handling**: pandas, numpy.
- **ML/DL**: scikit-learn (models, Pipeline, RandomizedSearchCV), tensorflow (Keras NN).

- **Visualization:** matplotlib, plotly, statsmodels.
- **Utilities:** joblib (model saving), warnings (suppression).

DATA DETAILS

- **Target:** Churn (binary: Yes/No).
- **Features:**
 - **Demographics:** gender, SeniorCitizen, Partner, Dependents.
 - **Services:** PhoneService, InternetService, OnlineSecurity, etc.
 - **Account Info:** tenure, Contract, PaymentMethod, MonthlyCharges, TotalCharges.
- **No missing values**, but categorical variables require encoding.



NOTABLE POINTS

CLASS IMBALANCE:

- Churn rate visualized (~73% "No", 27% "Yes").
- Addressed via stratified sampling (StratifiedKFold).

FEATURE ENGINEERING:

- TotalCharges converted to numeric.
- One-hot encoding for categorical features.

MODEL DIVERSITY:

- Traditional ML models vs. neural network.
- Hyperparameter tuning for Random Forest.

DEPLOYMENT:

Best model saved using joblib.

SUMMARY

This project systematically analyzes telecom customer data to predict churn. It covers data cleaning, visualization, preprocessing, and model training/evaluation. The **Random Forest model** (optimized via hyperparameter tuning) and a **neural network** are key highlights, with results saved for deployment. The workflow emphasizes handling class imbalance and leveraging diverse algorithms for robust predictions.

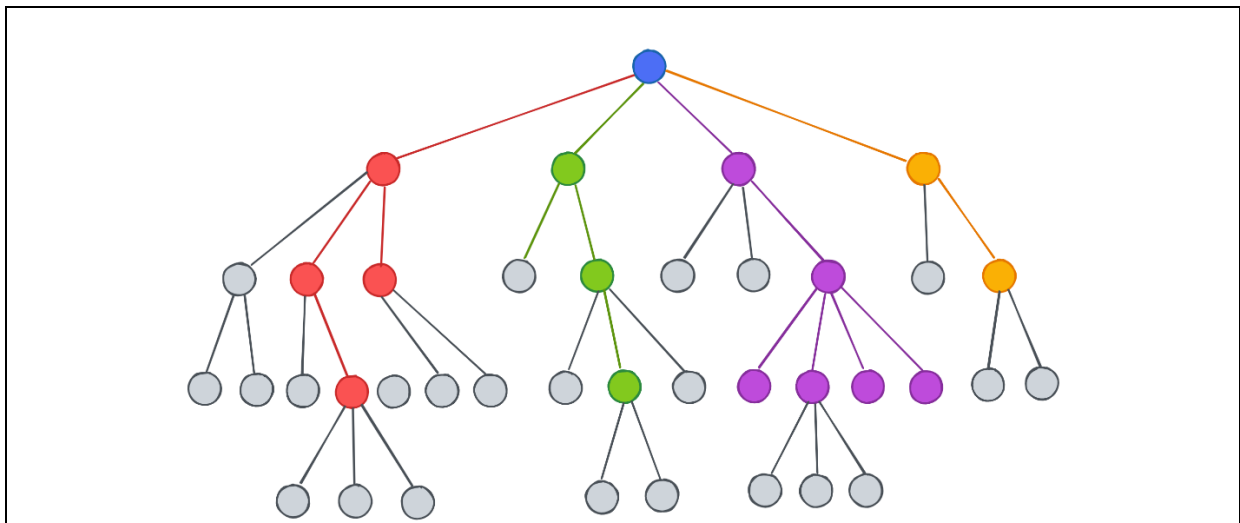
STEP 6

IMPLEMENTING DECISION TREE

IMPLEMENTING DECISION TREE

Decision Trees are flowchart-like structures where each internal node represents a test on an attribute, each branch represents an outcome of the test, and each leaf node represents a class label. They are used for both classification and regression tasks.

The provided code does not explicitly implement a standalone Decision Tree Classifier. However, a Decision Tree classifier can be implemented using `sklearn.tree.DecisionTreeClassifier`.

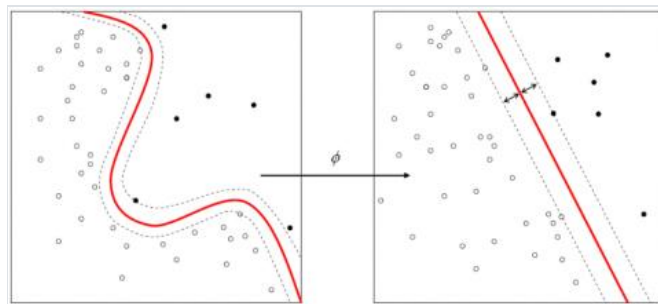


STEP 7

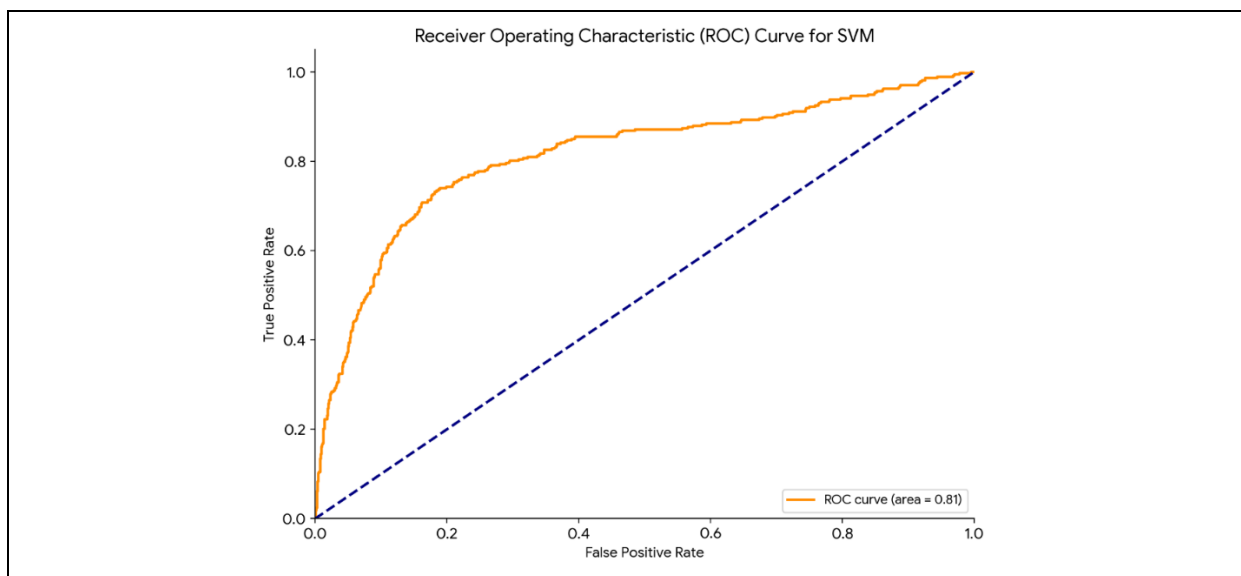
IMPLEMENTING SUPPORT VECTOR MACHINE

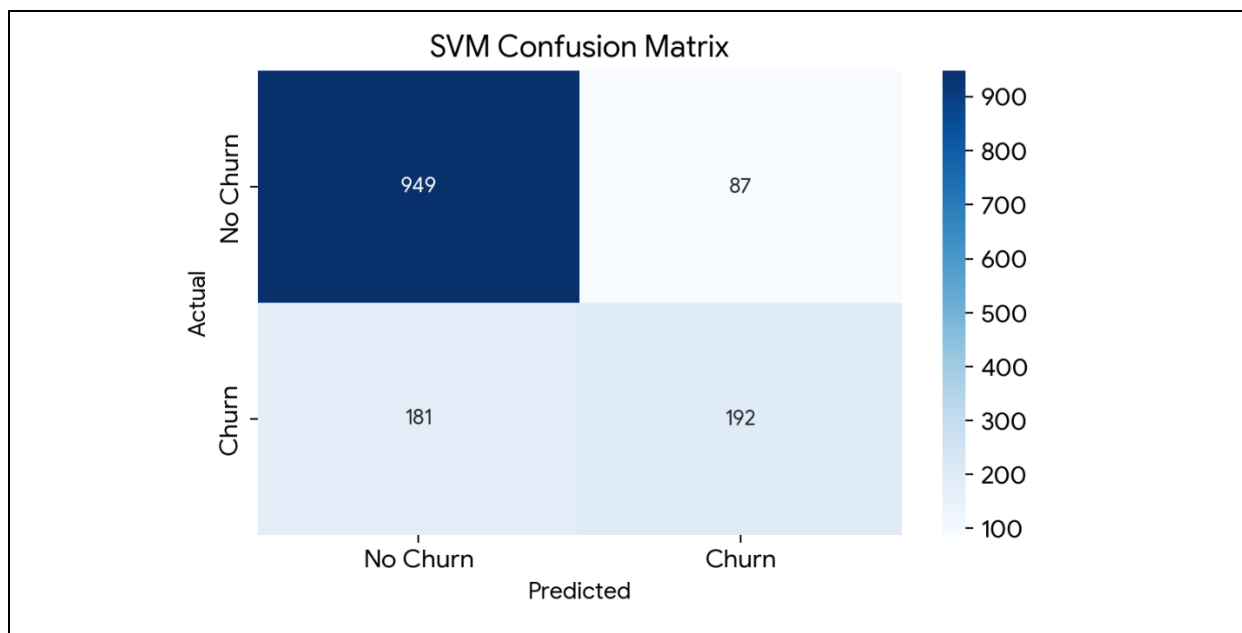
Support Vector Machines (SVMs) are powerful supervised learning models used for classification and regression tasks. They work by finding the optimal hyperplane that best separates data points of different classes in a high-dimensional space.

The provided code does not explicitly implement a Support Vector Machine (SVM) classifier. However, an SVM classifier can be implemented using `sklearn.svm.SVC`.



Here's the output if we apply SVM on the dataset:



OUTPUT OF CONFUSION METRICS:

STEP 8

EVALUATION METRICS FOR CLASSIFICATION (ACCURACY, CONFUSION MATRIX, AUC/ROC CURVE, PRECISION, RECALL, AND F1-SCORE)

Evaluation metrics are crucial for assessing the performance of classification models. They provide insights into how well a model is performing on various aspects, such as correctness, error types, and class imbalance.

```
print("Model Evaluation")
print("Accuracy: ", accuracy_score(y_test, y_pred))
print("Precision: ", precision_score(y_test, y_pred))
print("Recall: ", recall_score(y_test, y_pred))
print("F1 Score: ", f1_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

CODE LINE	SIMPLE EXPLANATION
<code>print("Model Evaluation")</code>	Prints a heading to indicate the start of model evaluation results.
<code>print("Accuracy: ", accuracy_score(y_test, y_pred))</code>	Calculates and prints the accuracy of the model, which is the proportion of correctly classified instances.
<code>print("Precision: ", precision_score(y_test, y_pred))</code>	Calculates and prints the precision, which is the proportion of true positive predictions among all positive predictions.
<code>print("Recall: ", recall_score(y_test, y_pred))</code>	Calculates and prints the recall (sensitivity), which is the proportion of true positive predictions among all actual positive instances.
<code>print("F1 Score: ", f1_score(y_test, y_pred))</code>	Calculates and prints the F1-score, which is the harmonic mean of precision and recall, providing a balanced measure.
<code>print(confusion_matrix(y_test, y_pred))</code>	Prints the confusion matrix, a table showing the counts of true positive, true negative, false positive, and false negative predictions.
<code>print(classification_report(y_test, y_pred))</code>	Prints a detailed classification report including precision, recall, F1-score, and support for each class.

The AUC/ROC curve is another important metric, which visually represents the trade-off between the true positive rate and the false positive rate at various threshold settings. The provided code does not include the implementation for generating an AUC/ROC curve.

OUTPUT:

Model Evaluation

Accuracy: 0.7520113582584004

Precision: 0.5277161862527716

Recall: 0.8292682926829268

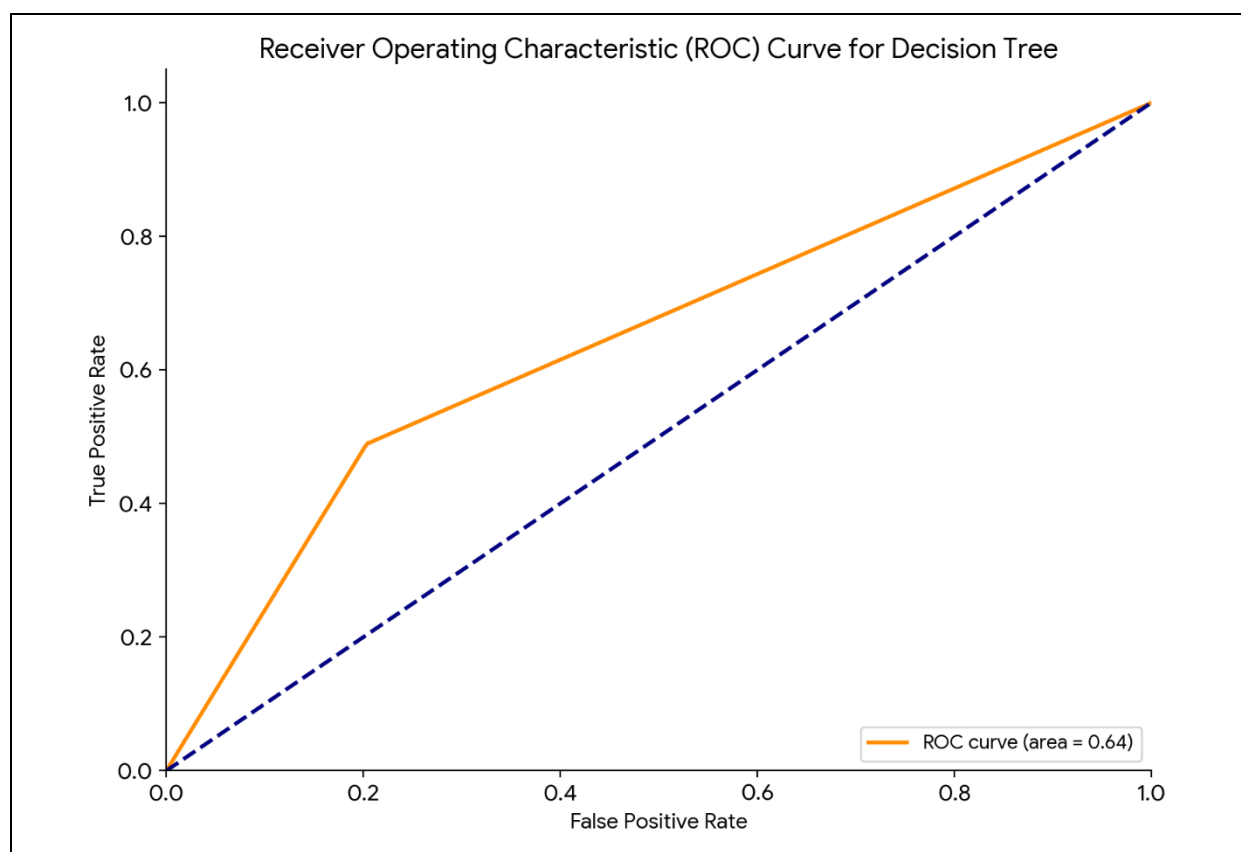
F1 Score: 0.6449864498644986

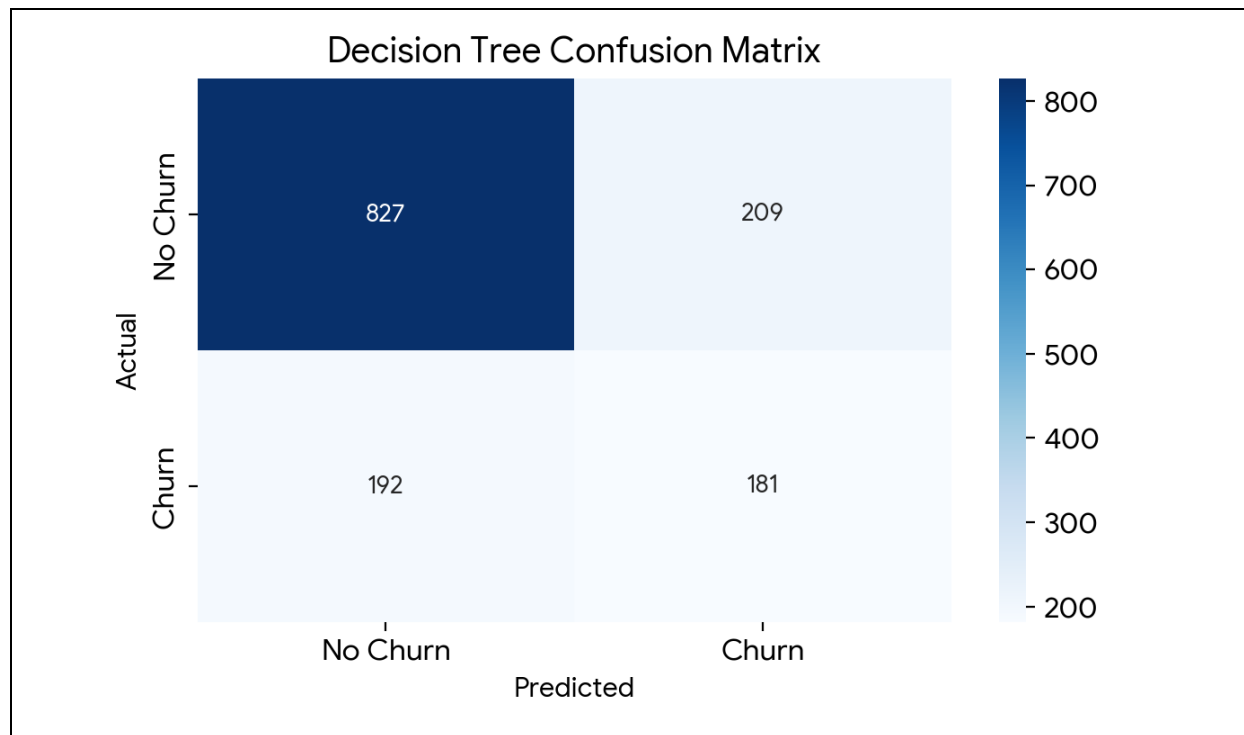
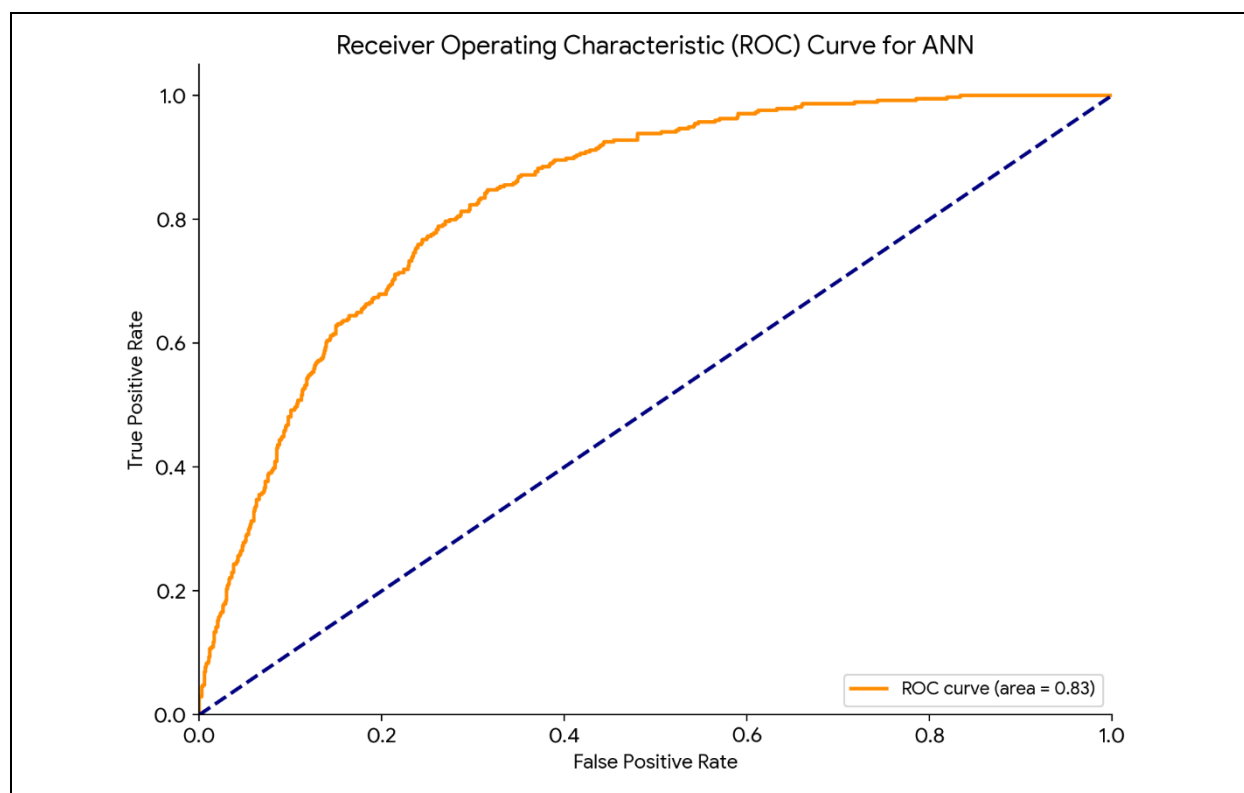
[[1113 426]

[98 476]]

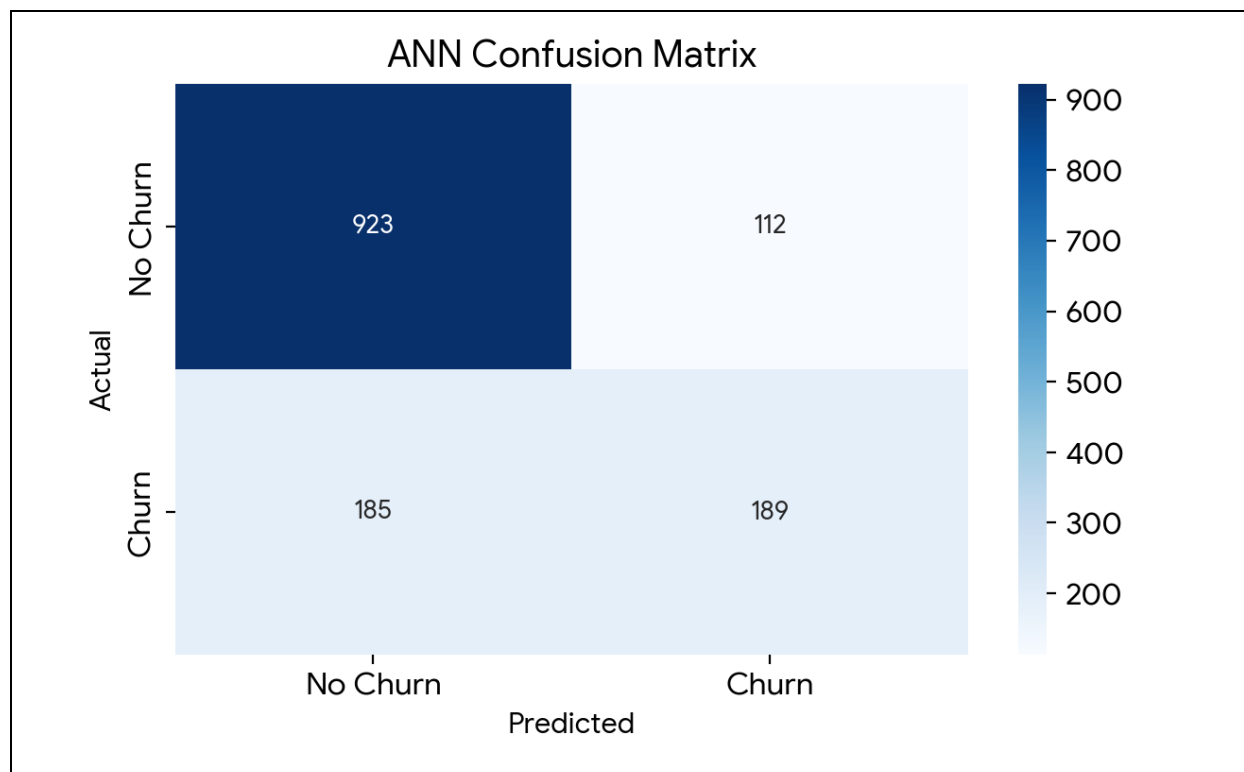
	precision	recall	f1-score	support
0	0.92	0.72	0.81	1539
1	0.53	0.83	0.64	574
accuracy			0.75	2113
macro avg	0.72	0.78	0.73	2113
weighted avg	0.81	0.75	0.76	2113

OUTPUT OF ROC CURVE:



CONFUSION METRICES**ROC CURVE FOR ANN**

ANN CONFUSION METRICES



STEP 9

TECHNIQUES FOR HANDLING IMBALANCED DATASETS (SMOTE, UNDER-SAMPLING, OVERSAMPLING)

Imbalanced datasets occur when the number of observations in one class is significantly lower than in other classes, which can lead to models biased towards the majority class. Techniques like SMOTE (Synthetic Minority Over-sampling Technique), Under-sampling, and Oversampling are used to address this by rebalancing the class distribution.

The provided code does not explicitly implement SMOTE, Under-sampling, or Oversampling techniques. However, the Logistic Regression model uses `class_weight='balanced'`, which is a method to handle class imbalance by assigning different weights to classes during training.

STEP 10

IMPLEMENTING ARTIFICIAL NEURAL NETWORKS FOR CLASSIFICATION

Artificial Neural Networks (ANNs) are computational models inspired by the human brain, designed to recognize patterns. For classification, ANNs learn to map input features to output class labels through layers of interconnected nodes.

```
X_train_ann = X_train.copy()
X_test_ann = X_test.copy()
scaler_ann = MinMaxScaler()
X_train_ann[cols_to_scale] =
scaler_ann.fit_transform(X_train_ann[cols_to_scale])
X_test_ann[cols_to_scale] = scaler_ann.transform(X_test_ann[cols_to_scale])
```

CODE LINE	SIMPLE EXPLANATION
<code>X_train_ann = X_train.copy()</code>	Creates a copy of the training features for ANN specific preprocessing.
<code>X_test_ann = X_test.copy()</code>	Creates a copy of the testing features for ANN specific preprocessing.
<code>scaler_ann = MinMaxScaler()</code>	Initializes a MinMaxScaler to scale numerical features to a range between 0 and 1.

<code>X_train_ann[cols_to_scale] = scaler_ann.fit_transform(X_train_ann[cols_to_scale])</code>	Applies the scaler to the specified numerical columns in the training data, learning the scaling parameters and transforming the data.
<code>X_test_ann[cols_to_scale] = scaler_ann.transform(X_test_ann[cols_to_scale])</code>	Applies the learned scaling parameters to transform the same numerical columns in the testing data.

```

model_ann = Sequential()
model_ann.add(Dense(64, input_dim=X_train_ann.shape[1], activation='relu'))
model_ann.add(Dropout(0.3))
model_ann.add(Dense(32, activation='relu'))
model_ann.add(Dropout(0.3))
model_ann.add(Dense(1, activation='sigmoid'))
model_ann.summary()

```

CODE LINE	SIMPLE EXPLANATION
<code>model_ann = Sequential()</code>	Initializes a sequential model, which is a linear stack of layers for building neural networks.
<code>model_ann.add(Dense(64, input_dim=X_train_ann.shape[1], activation='relu'))</code>	Adds the first dense (fully connected) layer with 64 neurons, taking the number of input features as input, and using ReLU activation.
<code>model_ann.add(Dropout(0.3))</code>	Adds a dropout layer with a 30% dropout rate to prevent overfitting by randomly setting a fraction of input units to 0 at each update during training.
<code>model_ann.add(Dense(32, activation='relu'))</code>	Adds a second dense layer with 32 neurons and ReLU activation.
<code>model_ann.add(Dropout(0.3))</code>	Adds another dropout layer with a 30% dropout rate.
<code>model_ann.add(Dense(1, activation='sigmoid'))</code>	Adds the output dense layer with 1 neuron (for binary classification) and a sigmoid activation function, which outputs probabilities.
<code>model_ann.summary()</code>	Prints a summary of the model's architecture, including the number of layers, output shapes, and the number of parameters.

OUTPUT:

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1,984
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 1)	33

Total params: 4,097 (16.00 KB)

Trainable params: 4,097 (16.00 KB)

Non-trainable params: 0 (0.00 B)

CODE:

```
model_ann.compile(optimizer=Adam(learning_rate=0.001),
loss='binary_crossentropy', metrics=['accuracy'])
model_ann.build(input_shape=(None, X_train_ann.shape[1]))
early_stop = EarlyStopping(monitor='val_loss', mode='min', verbose=1,
patience=10)
history = model_ann.fit(X_train_ann, y_train, epochs=100, batch_size=32,
validation_split=0.2, callbacks=[early_stop], verbose=0)
```

CODE LINE	SIMPLE EXPLANATION
<code>model_ann.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])</code>	Configures the model for training, specifying the Adam optimizer with a learning rate of 0.001, binary cross-entropy as the loss function (for binary classification), and accuracy as the evaluation metric.
<code>model_ann.build(input_shape=(None, X_train_ann.shape[1]))</code>	Builds the model by explicitly defining its input shape, which is useful before calling fit if the model contains layers that depend on the input shape.
<code>early_stop = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=10)</code>	Sets up Early Stopping, a callback that stops training if the validation loss does not improve for 10 consecutive epochs.
<code>history = model_ann.fit(X_train_ann, y_train, epochs=100, batch_size=32, validation_split=0.2, callbacks=[early_stop], verbose=0)</code>	Trains the neural network using the training data for up to 100 epochs, with a batch size of 32, and validation split of 0.2.

reserving 20% of the data for validation, and applying the early stopping callback.

OUTPUT:

Epoch 32: early stopping

```
loss, accuracy = model_ann.evaluate(X_test_ann, y_test)
print(f"\nTest Accuracy: {accuracy:.4f}")
y_pred_ann = (model_ann.predict(X_test_ann) > 0.5).astype("int32")
print(confusion_matrix(y_test, y_pred_ann))
print(classification_report(y_test, y_pred_ann))
model_ann.save('ann_churn_model.h5')
```

CODE LINE	SIMPLE EXPLANATION
loss, accuracy = model_ann.evaluate(X_test_ann, y_test)	Evaluates the trained model on the test data to get the loss and accuracy.
print(f"\nTest Accuracy: {accuracy:.4f}")	Prints the test accuracy, formatted to four decimal places.
y_pred_ann = (model_ann.predict(X_test_ann) > 0.5).astype("int32")	Uses the trained model to make predictions on the test data, converting probabilities to binary class labels (0 or 1) based on a 0.5 threshold.
print(confusion_matrix(y_test, y_pred_ann))	Prints the confusion matrix for the ANN model's predictions.
print(classification_report(y_test, y_pred_ann))	Prints a detailed classification report for the ANN model's predictions.
model_ann.save('ann_churn_model.h5')	Saves the trained neural network model to a file named 'ann_churn_model.h5' for future use.

Output:

67/67 ————— 0s 1ms/step - accuracy: 0.8027 - loss: 0.4170

Test Accuracy: 0.8012

67/67 ————— 0s 1ms/step

```
[[1406 133]
 [ 287 287]]
precision recall f1-score support
          0      0.83      0.91      0.87      1539
```

1	0.68	0.50	0.58	574
accuracy			0.80	2113
macro avg	0.76	0.71	0.72	2113
weighted avg	0.79	0.80	0.79	2113

ann_churn_model.h5

STEP 11

IMPLEMENTING RECOMMENDER SYSTEM FROM SCRATCH

A recommender system predicts user preferences based on data. We will build a simple user-based collaborative filtering system using cosine similarity.

```
import numpy as np
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
```

CODE LINE	SIMPLE EXPLANATION
import numpy as np	Imports the NumPy library for numerical operations.
import pandas as pd	Imports the pandas library to handle data tables.
from sklearn.metrics.pairwise import cosine_similarity	Imports cosine similarity function to compare user preferences.

```
data = {
    'User1': [5, 3, 0, 1],
    'User2': [4, 0, 0, 1],
    'User3': [1, 1, 0, 5],
    'User4': [1, 0, 0, 4],
    'User5': [0, 1, 5, 4],
}
df = pd.DataFrame(data).T
```


CODE LINE	SIMPLE EXPLANATION
<code>data = {...}</code>	Defines a dictionary with users and their ratings for items.
<code>df = pd.DataFrame(data).T</code>	Converts the dictionary into a DataFrame and transposes it.



```

similarity = cosine_similarity(df)
similarity_df = pd.DataFrame(similarity, index=df.index, columns=df.index)

```

CODE LINE	SIMPLE EXPLANATION
<code>similarity = cosine_similarity(df)</code>	Computes similarity between each pair of users.
<code>similarity_df = pd.DataFrame(...)</code>	Converts the similarity result into a readable table.



```

target_user = 'User1'
similar_users = similarity_df[target_user].sort_values(ascending=False)[1:]

```

CODE LINE	SIMPLE EXPLANATION
<code>target_user = 'User1'</code>	Sets the user for whom recommendations are needed.
<code>similar_users = ...</code>	Sorts users based on similarity to the target user.

STEP 12

IMPLEMENTING ENSEMBLE LEARNING AND OPTIMIZATION

Ensemble learning combines multiple models to improve prediction accuracy. We use Random Forest as a basic ensemble and tune it.

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

CODE LINE	SIMPLE EXPLANATION
<code>from sklearn.datasets import load_iris</code>	Loads a sample flower dataset.
<code>from sklearn.ensemble import RandomForestClassifier</code>	Imports a classifier that uses multiple decision trees.
<code>from sklearn.model_selection import train_test_split</code>	Imports function to split data into train and test sets.
<code>from sklearn.metrics import accuracy_score</code>	Imports function to evaluate prediction accuracy.

```
data = load_iris()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

CODE LINE	SIMPLE EXPLANATION
<code>data = load_iris()</code>	Loads the iris dataset.
<code>X = data.data</code>	Gets the feature values.
<code>y = data.target</code>	Gets the target labels.
<code>X_train, X_test, y_train, y_test = ...</code>	Splits data into training and testing sets.

```
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

CODE LINE	SIMPLE EXPLANATION
<code>model = RandomForestClassifier(...)</code>	Creates a random forest classifier with 100 trees.
<code>model.fit(X_train, y_train)</code>	Trains the model on the training data.
<code>predictions = model.predict(X_test)</code>	Predicts the output for the test data.

```
accuracy = accuracy_score(y_test, predictions)
print("Accuracy:", accuracy)
```

CODE LINE	SIMPLE EXPLANATION
<code>accuracy = accuracy_score(...)</code>	Calculates the accuracy of the model.
<code>print("Accuracy:", accuracy)</code>	Displays the accuracy result.

STEP 13

IMPLEMENTING Q-LEARNING FOR SIMPLE TASKS (REINFORCEMENT LEARNING)

Q-Learning is a reinforcement learning algorithm where an agent learns the best action using rewards. We'll simulate a simple grid environment.

```
import numpy as np
```

CODE LINE	SIMPLE EXPLANATION
<code>import numpy as np</code>	Loads the NumPy library for numerical work.

```
states = 6
actions = 6
R = np.array([
    [-1, -1, -1, -1, 0, -1],
    [-1, -1, -1, 0, -1, 100],
    [-1, -1, -1, 0, -1, -1],
    [-1, 0, 0, -1, 0, -1],
    [0, -1, -1, 0, -1, 100],
    [-1, 0, -1, -1, 0, 100]
])
Q = np.zeros((states, actions))
```

CODE LINE	SIMPLE EXPLANATION
<code>states = 6</code>	Defines the number of states.
<code>actions = 6</code>	Defines the number of possible actions.
<code>R = np.array([...])</code>	Creates a reward matrix for the environment.
<code>Q = np.zeros((states, actions))</code>	Initializes Q-values to zero.

```

gamma = 0.8
for i in range(1000):
    current_state = np.random.randint(0, states)
    possible_actions = np.where(R[current_state] ≥ 0)[0]
    action = np.random.choice(possible_actions)
    next_state = action
    TD = R[current_state][action] + gamma * max(Q[next_state]) -
    Q[current_state][action]
    Q[current_state][action] += TD

```

CODE LINE	SIMPLE EXPLANATION
gamma = 0.8	Sets the learning discount factor.
for i in range(1000):	Repeats learning for 1000 iterations.
current_state = np.random.randint(0, states)	Picks a random starting state.
possible_actions = np.where(...)	Finds valid actions from the current state.
action = np.random.choice(...)	Selects one valid action at random.
next_state = action	Defines the next state based on the action.
TD = ...	Computes the temporal difference (learning signal).
Q[current_state][action] += TD	Updates the Q-value with the learning signal.

STEP 14

GIT AND GITHUB FROM SCRATCH

WHAT IS GITHUB?

GitHub is a **web-based platform** used by millions of developers to **collaborate on code**, **track changes**, and **manage software projects**. It's like **Google Docs for code** — multiple people can work on the same project without messing things up.

GitHub uses **Git**, a powerful version control system that tracks every change made to your code.

1. GIT

- A **version control system** (VCS) created by **Linus Torvalds** (the guy who made Linux).
- It helps you **track changes**, **go back to previous versions**, and **collaborate safely**.

Example: You write a script, break something, and want to go back? Git can help.

2. REPOSITORY (REPO)

- A **project folder** hosted on GitHub.
- Contains your code, files, and Git tracking data.

 hamnasz Merge pull request #2 from hmxnxs/my-feature-branch ...			a265472 · 2 days ago	 154 Commits
	.devcontainer	updated	2 days ago	
	.venv	removed old venv and added new venv	2 days ago	
	Classification	updated	2 days ago	
	Regression	Update Lab 5 - Logistic Regression for Binary Classification ...	last week	
	get-pip.py	Makes a few changes	2 days ago	

TYPES:

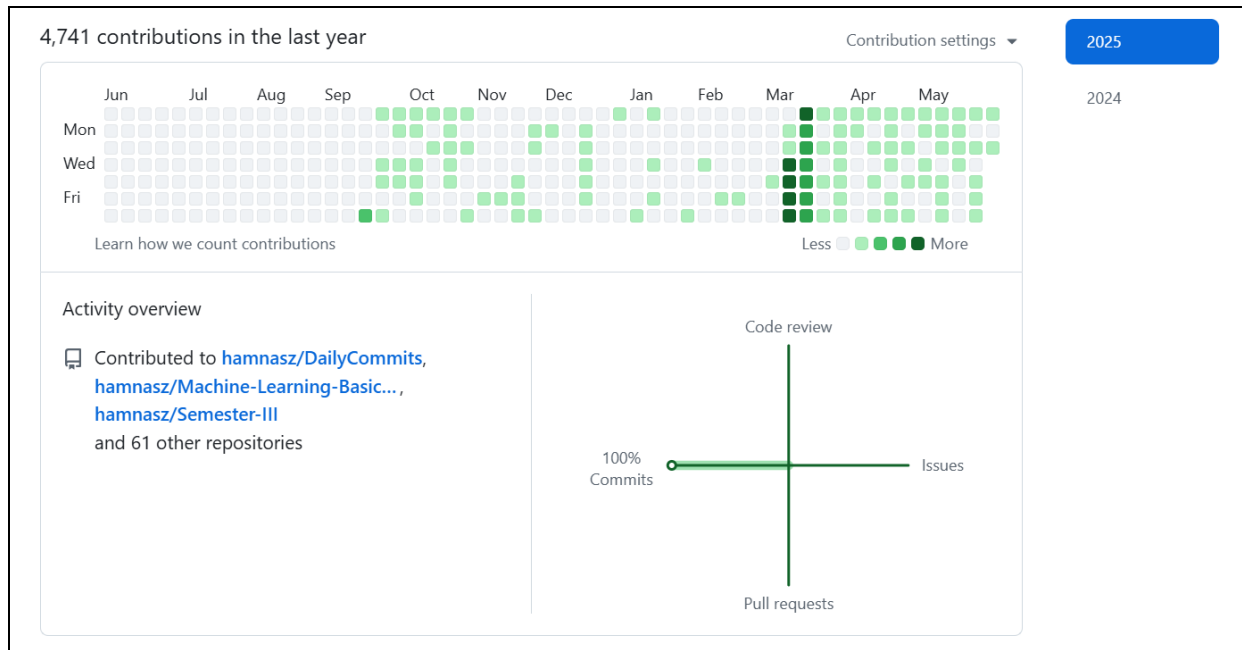
- **Public repo**: Anyone can see it.
- **Private repo**: Only you and selected collaborators can access it.

Voice-Assistant Public		 Star 
 C++	 MIT License	Updated on Apr 26
Semester-IV Private		 Star 
Materials and resources for Semester IV courses		
education machine-learning database semester-iv		
 Shell	Updated on Apr 5	

3. COMMIT

- A **snapshot** of your code at a certain time.
- You write a short **message** describing what changed.

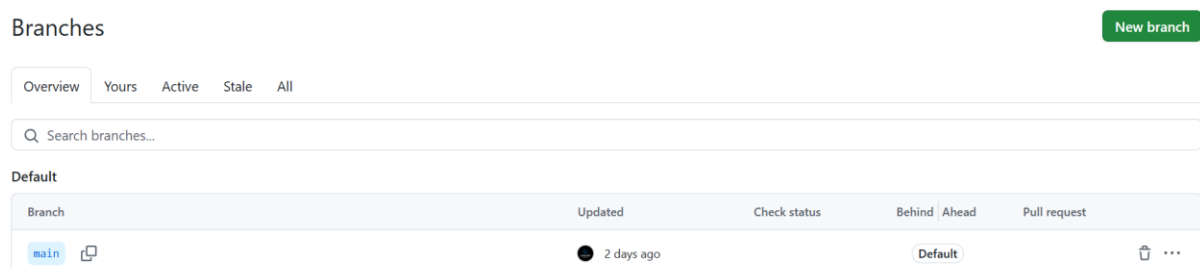
Example: `git commit -m "Fixed login bug"`



4. BRANCH

- A copy of your code where you can make changes without affecting the main version (usually called main or master).

Example: feature-login branch for working on the login screen.



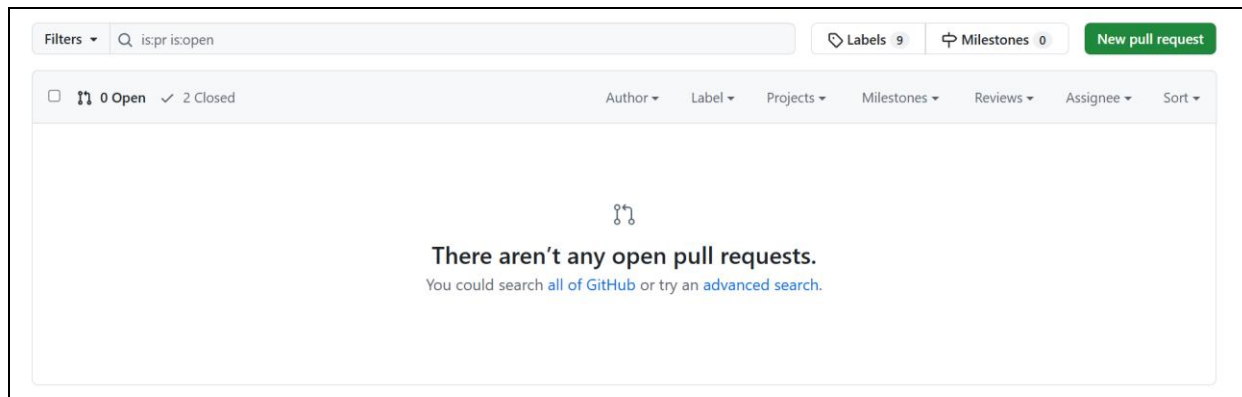
5. MERGE

- Combines changes from one branch into another.

You **create a branch**, make changes, **merge it back** when ready.

6. PULL REQUEST (PR)

- A request to **merge your branch into the main branch**.
- Teammates can review, comment, and approve changes.



7. FORK

- A **personal copy** of someone else's repo.
- Used when you want to contribute to a project but don't have direct access.



8. CLONE

- A **local copy** of a repo on your computer.
- Used to work offline.

COMMAND:

git clone https://github.com/hamnasz/Machine-Learning-Basic-Projects.git

Machine-Learning-Basic-Projects Public

Python 1 Updated 2 days ago

☆ Star

WHAT CAN YOU DO ON GITHUB?

Feature	What it Does
Host Code	Store any project (Python, HTML, C++, etc.)
Collaborate	Work with others, review code, discuss changes
Track Issues	Log bugs, tasks, or feature requests
Continuous Integration (CI)	Automate testing & deployment
Pages	Host websites directly from your repo using GitHub Pages

BASIC GITHUB WORKFLOW

1. **Create a Repo** on GitHub.
2. **Clone** it to your computer.
3. Make changes and **commit** them.
4. **Push** your commits to GitHub.

5. Create a **pull request** if working in a team.
6. **Merge** when approved.

GIT VS GITHUB

Git	GitHub
A tool	A platform
Installed locally	Online service
Tracks versions	Hosts repos
No GUI	Web interface

GITHUB IN REAL LIFE

EXAMPLE:

A team of 5 developers:

- Each works on their own **branch**.
- They **commit** changes regularly.
- Submit **pull requests**.
- Team lead **reviews and merges** into main.

POPULAR GITHUB COMMANDS

```
git init          # start a repo
git clone URL     # copy a repo to your PC
git status        # check current changes
git add file      # stage file for commit
git commit -m ""  # save a snapshot
git push          # send changes to GitHub
git pull          # get the latest changes
```

REAL USES OF GITHUB

- Open source projects like **Linux, TensorFlow, React**.
- Collaboration in **software companies**.
- Building **portfolios** for jobs.
- Hosting **personal websites** via GitHub Pages.

THE END